

Document Number: P0244R2

Date: 2017-06-13

Audience: Library Evolution Working Group

Reply-to: Tom Honermann <tom@honermann.net>

Text_view: A C++ concepts and range based character encoding and code point enumeration library

- [Changes Since P0244R1](#)
- [Introduction](#)
- [Motivation and Scope](#)
- [Terminology](#)
 - [Code Unit](#)
 - [Code Point](#)
 - [Character Set](#)
 - [Character](#)
 - [Encoding](#)
- [Design Considerations](#)
 - [View Requirements](#)
 - [Error Handling](#)
 - [Encoding Forms vs Encoding Schemes](#)
 - [Streaming](#)
 - [Character Types](#)
 - [Locale Dependent Encodings](#)
- [Implementation Experience](#)
- [Future Directions](#)
 - [Transcoding](#)
 - [Constexpr Support](#)
 - [Unicode Normalization Iterators](#)
 - [Unicode Grapheme Cluster Iterators](#)
- [FAQ](#)
 - [Why do I have to specify the encoding for UTF-8 string literals?](#)
 - [Can I define my own encodings? If so, How?](#)
- [Technical Specifications](#)
 - [Header <experimental/text_view> synopsis](#)
 - [Concepts](#)
 - [Error Policies](#)
 - [Error Status](#)
 - [Exceptions](#)
 - [Type Traits](#)
 - [Character Sets](#)
 - [Character Set Identification](#)
 - [Character Set Information](#)
 - [Characters](#)
 - [Encodings](#)
 - [Text Iterators](#)
 - [Text View](#)
- [Acknowledgements](#)
- [References](#)

Changes Since P0244R1

Major changes

- Added support for error handling policies to allow choosing how encoding and decoding errors are communicated. Two policies have been introduced:
 - **strict** (default)
Exceptions are thrown for attempts to encode an invalid character or to retrieve a character corresponding to an ill-formed code unit sequence (exceptions are not thrown when the ill-formed code unit sequence is first decoded).
 - **permissive**
No exceptions are thrown for attempts to encode an invalid character or to decode an ill-formed code unit sequence. Instead,

substitution characters are encoded in place of invalid characters or produced in place of ill-formed code unit sequences.

- Updated text iterator requirements to enable checking for errors before performing an operation that might throw an exception. For example:

```
const char s[] = "\xC0"; // ill-formed UTF-8 code unit sequence.
auto tv = make_text_view<char8_character_encoding>(s);
auto ti = tv.begin();
if (ti.error_occurred()) {
    decode_status ds = ti.get_error();
}
*ti; // would throw text_decode_error.
```

- Added support for assuming a default encoding for calls to `make_text_view` that do not specify an explicit encoding. Default encodings are assumed based on the code unit type of the range type provided in the call as follows.

Code unit type	Default encoding
<code>char</code>	<code>execution_character_encoding</code>
<code>wchar_t</code>	<code>execution_wide_character_encoding</code>
<code>char16_t</code>	<code>char16_character_encoding</code>
<code>char32_t</code>	<code>char32_character_encoding</code>

Note that support for assuming the `char8_character_encoding` is not present. Such support would require a unique code unit type as proposed in [P0482R0](#) [\[P0482R0\]](#).

Detailed changes

- Updated the FAQ [Why do I have to specify the encoding for UTF-8 string literals?](#) section and added references to [P0482R0](#) [\[P0482R0\]](#).
- Error handling changes:
 - Updated the [Error Handling](#) section with a description of the new error handling behavior.
 - Added the `TextErrorPolicy` concept, `text_error_policy` base class, `text_strict_error_policy` and `text_permissive_error_policy` error policy classes, and the `text_default_error_policy` alias. Modified `itext_iterator`, `itext_sentinel`, `otext_iterator`, and `basic_text_view` to accept an error policy template parameter. Added `make_otext_iterator` and `make_text_view` overloads to enable construction of text views with a particular error policy.
 - Removed the `TextDecoder` concept and modified the `decode` and `rdecode` member functions of all codecs to require iterators that satisfy `ForwardIterator`. In general, it isn't possible to implement the `decode` and `rdecode` interfaces for input iterators and support reasonable error recovery.
 - Added `noexcept` requirements and conditional `noexcept` specifications for encode and decode operations.
 - Added the `encode_status` and `decode_status` enum classes to support error handling without exceptions.
 - Modified the `encode_state_transition` and `encode` member functions required by the `TextEncoder` concept to return a value of type `encode_status` rather than `void`.
 - Modified the `decode` and `rdecode` member functions required by the `TextForwardDecoder` and `TextBidirectionalDecoder` concepts to return a value of type `decode_status` rather than `bool`. A return value of `decode_status::no_error` corresponds to a prior return of `true` and a value of `decode_status::no_character` corresponds to a prior return of `false`.
 - Added `error_occurred` and `get_error` member function requirements to the text iterator concepts.
 - Renamed the `text_runtime_error` exception class to `text_error`.
 - Modified the `text_encode_error` and `text_decode_error` classes to require an `encode_status` or `decode_status` value on construction and to make it available via a `status_code` member function.
 - Removed the `text_encode_overflow_error` and `text_decode_underflow_error` exception classes.
- Added `default_encoding_type_t` to allow assuming an encoding type based on code unit type. Added corresponding `make_text_view` overloads.
- Updated the `CharacterSet` concept to require a `get_substitution_code_point` static member function and added implementations to `any_character_set`, `basic_execution_character_set`, `basic_execution_wide_character_set`, and `unicode_character_set`.
- Updated the `Character` concept to require constructibility from a code point type.
- Added concepts for text iterators and views to allow specifying iterator category constraints more conveniently.
- Updated the `TextIterator`, `TextOutputIterator`, and `TextInputIterator` concepts to allow `TextOutputIterator` to refine `TextIterator`. The requirement for a text iterator's value type to satisfy `Character` has been moved to `TextInputIterator`.
- Removed the requirement for non-const access to the underlying view via a `base()` member function for `TextView`; removed the corresponding member function from `basic_text_view`.
- Updated the `Character` concept to remove the redundant `Copyable` requirement; `Regular` subsumes `Copyable`. Thanks to Casey Carter for spotting this.
- Updated the base member functions of `itext_iterator`, `itext_sentinel`, and `otext_iterator` to return const references rather than copies.
- Removed the constraint that the `base_range()` member function of `itext_iterator` be present only for forward views. Caching is now required for input views.
- Added the `look_ahead_range()` member function to `itext_iterator` to enable retrieving code units that were consumed from the

underlying input iterator, but not used to decode a character. Such consumption happens in error scenarios.

- Removed equality comparison operators for `itext_sentinel`. Equality comparison requirements for sentinels was removed from the Ranges proposal in N4569.
- Updated view types to derive from `ranges::view_base`.
- Updated a few `basic_text_view` constructors to require underlying view construction from pairs of rvalue iterators. This was done so that implementations can make use of move semantics to forward arguments.
- Changes to reflect the range-based for statement enhancements provided by P0184R0 as adopted for C++17.

Introduction

C++11 [\[C++11\]](#) added support for new character types [\[N2249\]](#) and Unicode string literals [\[N2442\]](#), but neither C++11, nor more recent standards have provided means of efficiently and conveniently enumerating code points in Unicode or legacy encodings. While it is possible to implement such enumeration using interfaces provided in the standard `<locale>` and `<codecvt>` libraries, doing so is awkward, requires that text be provided as pointers to contiguous memory, and inefficient due to virtual function call overhead.

The described library provides iterator and range based interfaces for encoding and decoding strings in a variety of character encodings. The interface is intended to support all modern and legacy character encodings, though implementations are expected to only provide support for a limited set of encodings.

An example usage follows. Note that `\u00F8` (LATIN SMALL LETTER O WITH STROKE) is encoded as UTF-8 using two code units (`\xC3\xB8`), but iterator based enumeration sees just the single code point.

```
using CT = utf8_encoding::character_type;
auto tv = make_text_view<utf8_encoding>(u8"J\u00F8erg");
auto it = tv.begin();
assert(*it++ == CT{0x004A}); // 'J'
assert(*it++ == CT{0x00F8}); // 'ø'
assert(*it++ == CT{0x0065}); // 'e'
```

The provided iterators and views are compatible with the non-modifying sequence utilities provided by the standard C++ `<algorithm>` library. This enables use of standard algorithms to search encoded text.

```
it = std::find(tv.begin(), tv.end(), CT{0x00F8});
assert(it != tv.end());
```

The iterators also provide access to the underlying code unit sequence.

```
auto base_it = it.base_range().begin();
assert(*base_it++ == '\xC3');
assert(*base_it++ == '\xB8');
assert(base_it == it.base_range().end());
```

These ranges satisfy the requirements for use in C++11 range-based for statements with the removed same type restriction for the `begin` and `end` expressions provided by P0184R0 [\[P0184R0\]](#) as adopted for C++17.

```
for (const auto& ch : tv) {
    ...
}
```

`make_text_view` overloads are provided that assume an encoding based on code unit type for code unit types that imply an encoding. Note that it is currently not possible to assume an encoding for UTF-8 string literals. See the [FAQ entry](#) regarding this for more details.

```
auto char_tv = make_text_view("text");
static_assert(std::is_same<
    encoding_type_t<decltype(char_tv)>,
    execution_character_encoding::value>);
```

Motivation and Scope

Consider the following code to search for the occurrence of U+00F8 in the UTF-8 encoded string using C++ standard provided interfaces.

```

std::string s = u8"J\u00F8erg";
std::mbstate_t state = std::mbstate_t{};
codecvt_utf8<char32_t> utf8_converter;
const char *from_begin = s.data();
const char *from_end = s.data() + s.size();
const char *from_current;
const char *from_next = from_begin;
char32_t to[1];
std::codecvt_base::result r;
do {
    from_current = from_next;
    char32_t *to_begin = &to[0];
    char32_t *to_end = &to[1];
    char32_t *to_next;
    r = utf8_converter.in(
        state,
        from_current, from_end, from_next,
        to_begin, to_end, to_next);
} while (r != std::codecvt_base::error && to[0] != char32_t{0x00F8});
if (r != std::codecvt_base::error && to[0] == char32_t{0x00F8}) {
    cout << "Found at offset " << (from_current - from_begin) << endl;
} else {
    cout << "Not found" << endl;
}

```

There are a number of issues with the above code:

- It is verbose.
- It is limited to working with strings that are stored in contiguous memory.
- It is inefficient. All `codecvt` public member functions dispatch to virtual member functions.
- It is not generic. Use of the `codecvt_utf8` facet makes it specific to handling of UTF-8 encoded text. Making this code generic would require some other means of identifying an appropriate facet to use.
- It is not applicable to non-Unicode encodings; `codecvt` doesn't provide means to retrieve a code point for the encodings used for ordinary and wide strings. The above code only accomplishes this by depending on transcoding to UTF-32 and the fact that UTF-32 is a trivial encoding.

The above method is not the only method available to identify a search term in an encoded string. For some encodings, it is feasible to encode the search term in the encoding and to search for a matching code unit sequence. This approach works for UTF-8, UTF-16, and UTF-32 (assuming the search term and text to search are similarly normalized), but not for many other encodings. Consider the Shift-JIS encoding of U+6D6C. This is encoded as 0x8A 0x5C. Shift-JIS is a multibyte encoding that is almost ASCII compatible. The code unit sequence 0x5C encodes the ASCII `'\'` character. But note that 0x5C appears as the second byte of the code unit sequence for U+6D6C. Naively searching for the matching code unit sequence for `'\'` would incorrectly match the trailing code unit sequence for U+6D6C.

The library described here is intended to solve the above issues while also providing a modern interface that is intuitive to use and can be used with other standard provided facilities; in particular, the C++ standard `<algorithm>` library.

Terminology

The terminology used in this document is intended to be consistent with industry standards and, in particular, the Unicode standard. Any inconsistencies in the use of this terminology and that in the Unicode standard is unintentional. The terms described in this document comprise a subset of the terminology used within the Unicode standard; only those terms necessary to specify functionality exhibited by the proposed library are included here. Those who would like to learn more about general text processing terminology in computer systems are encouraged to read chapter 2, "General Structure" of the Unicode standard.

Code Unit

A single, indivisible, integral element of an encoded sequence of characters. A sequence of one or more code units specifies a code point or encoding state transition as defined by a character encoding. A code unit does not, by itself, identify any particular character or code point; the meaning ascribed to a particular code unit value is derived from a character encoding definition.

The `char`, `wchar_t`, `char16_t`, and `char32_t` types are most commonly used as code unit types.

The string literal `u8"J\u00F8erg"` contains 7 code units and 6 code unit sequences; `"\u00F8"` is encoded in UTF-8 using two code units and string literals contain a trailing NUL code unit.

The string literal `"J\u00F8erg"` contains an implementation defined number of code units. The standard does not specify the encoding of ordinary and wide string literals, so the number of code units encoded by `"\u00F8"` depends on the implementation defined encoding used for ordinary string literals.

Code Point

An integral value denoting an abstract character as defined by a character set. A code point does not, by itself, identify any particular character; the meaning ascribed to a particular code point value is derived from a character set definition.

The `char`, `wchar_t`, `char16_t`, and `char32_t` types are most commonly used as code point types.

The string literal `u8"J\u00F8erg"` describes a sequence of 6 code point values; string literals implicitly specify a trailing NUL code point.

The string literal `"J\u00F8erg"` describes a sequence of an implementation defined number of code point values. The standard does not specify the encoding of ordinary and wide string literals, so the number of code points encoded by `"\u00F8"` depends on the implementation defined encoding used for ordinary string literals. Implementations are permitted to translate a single code point in the source or Unicode character sets to multiple code points in the execution encoding.

Character Set

A mapping of code point values to abstract characters. A character set need not provide a mapping for every possible code point value representable by the code point type.

C++ does not specify the use of any particular character set or encoding for ordinary and wide character and string literals, though it does place some restrictions on them. Unicode character and string literals are governed by the Unicode standard.

Common character sets include ASCII, Unicode, and Windows code page 1252.

Character

An element of written language, for example, a letter, number, or symbol. A character is identified by the combination of a character set and a code point value.

Encoding

A method of representing a sequence of characters as a sequence of code unit sequences.

An encoding may be stateless or stateful. In stateless encodings, characters may be encoded or decoded starting from the beginning of any code unit sequence. In stateful encodings, it may be necessary to record certain affects of previously encoded characters in order to correctly encode additional characters, or to decode preceding code unit sequences in order to correctly decode following code unit sequences.

An encoding may be fixed width or variable width. In fixed width encodings, all characters are encoded using a single code unit sequence and all code unit sequences have the same length. In variable width encodings, different characters may require multiple code unit sequences, or code unit sequences of varying length.

An encoding may support bidirectional or random access decoding of code unit sequences. In bidirectional encodings, characters may be decoded by traversing code unit sequences in reverse order. Such encodings must support a method to identify the start of a preceding code unit sequence. In random access encodings, characters may be decoded from any code unit sequence within the sequence of code unit sequences, in constant time, without having to decode any other code unit sequence. Random access encodings are necessarily stateless and fixed length. An encoding that is neither bidirectional, nor random access, may only be decoded by traversing code unit sequences in forward order.

An encoding may support encoding characters from multiple character sets. Such an encoding is either stateful and defines code unit sequences that switch the active character set, or defines code unit sequences that implicitly identify a character set, or both.

A trivial encoding is one in which all encoded characters correspond to a single character set and where each code unit encodes exactly one character using the same value as the code point for that character. Such an encoding is stateless, fixed width, and supports random access decoding.

Common encodings include the Unicode UTF-8, UTF-16, and UTF-32 encodings, the ISO/IEC 8859 series of encodings including ISO/IEC 8859-1, and many trivial encodings such as Windows code page 1252.

Design Considerations

View Requirements

The `basic_text_view` and `itext_iterator` class templates are parameterized on a view type that provides access to the underlying code unit sequence. `make_text_view` and the various type aliases of `basic_text_view` are required to choose a view type to select a specialization of these class templates. The C++ standard library doesn't currently define a suitable view type, though the need for one has been recognized. N3350 [\[N3350\]](#) proposed a `std::range` class template to fill this need and the ranges proposal [\[N4560\]](#) states (C.2, "Iterator Range Type") that a future paper will propose such a type.

The technical specification in this paper leaves the view type selected by `make_text_view` and the type aliases of `basic_text_view` up to the implementation. It would have been possible to define a suitable view type as part of this library, but the author felt it better to wait until a suitable type becomes available as part of either the ranges proposal or the standard library.

Error Handling

Since use of exceptions is not acceptable to many members of the C++ community, this library supports multiple methods of error handling.

The low level encoding and decoding operations performed by the `encode_state_transition()`, `encode()`, `decode()`, and `rdecode()` static member functions required by the text encoding concepts return error indicators, do not directly throw exceptions, but allow exceptions to propagate as a result of exceptions thrown by operations performed on the provided code unit iterators. If the relevant advancement and dereference operators of the code unit iterators are `noexcept`, then these functions are also declared `noexcept`. Calls to these functions require explicit error checking.

By default, text iterators throw exceptions for errors that occur during encoding and decoding operations. Exceptions are only thrown (assuming non-throwing code unit iterators) during iterator dereference (for input text iterators) and dereference assign (for output text iterators); exceptions are not thrown when advancing text iterators (again, subject to the base code unit iterators having non-throwing operators). For text input iterators, this implies that errors encountered during advancement are held within these iterators until a dereference operation is performed. This approach has three benefits:

1. Following advancement of a text input iterator, the iterator is still in a valid state, information about the error is available, and the presumably invalid code unit sequence that resulted in the error is available for inspection prior to attempting to retrieve a decoded character.
2. A text input iterator can be advanced beyond an invalid code unit sequence. (The usual requirement to invoke the dereference operator following advancement of an input iterator is waived for text iterators). This implies that the low level decode operations must have means to advance beyond the invalid code unit sequence and to identify the start of the next potentially well formed sequence.
3. Exceptions will not be thrown upon construction of a text iterator or when calling `begin()` for a text view. Implicit advancement occurs on construction of a text input iterator as required to consume leading non-character encoding code unit sequences so that an iterator produced by calling `begin()` on a text view will compare equally to a corresponding `end()` iterator. (Consider a UTF encoded string that contains only a BOM).

Text iterators and views allow specifying an error handling policy via a template parameter. Two error policies are provided:

- Strict (`text_strict_error_policy`)
This is the default error policy that results in exceptions being thrown as described above.
- Permissive (`text_permissive_error_policy`)
The permissive error policy avoids exceptions by substituting characters (for example, the Unicode U+FFFD replacement character) when errors occur. During encoding operations, if the encode operation fails, for example with an invalid character error, an attempt will be made to encode the substitution character instead (if that fails, no character may be encoded). During decode operations, dereference operations that would have resulted in an exception being thrown will instead provide the substitution character as if it had been decoded. Error information is still retained for explicit inspection in this case.

Encoding Forms vs Encoding Schemes

The Unicode standard differentiates code unit oriented and byte oriented encodings. The former are termed encoding forms; the latter, encoding schemes. This library provides support for some of each. For example, `utf16_encoding` is code unit oriented; the value type of its iterators is `char16_t`. The `utf16be_encoding`, `utf16le_encoding`, and `utf16bom_encoding` encodings are byte oriented; the value type of their iterators is `char`.

Streaming

Decoding from a streaming source without unacceptably blocking on underflow requires the ability to decode a partial code unit sequence, save state, and then resume decoding the remainder of the code unit sequence when more data becomes available. This requirement presents challenges for an iterator based approach. The specification presented in this paper does not provide a good solution for this use case.

One possibility is to add additional state tracking that is stored with each iterator. Support for the possibility of trailing non-code-point encoding code unit sequences (escape sequences in some encodings) already requires that code point iterators greedily consume code units. This enables an iterator to compare equal to the end iterator even when its current base code unit iterator does not equal the end iterator of the underlying code unit range. Storing partial code unit sequence state with an iterator that compares equal to the end iterator would enable users to write code like the following.

```
using encoding = utf8_encoding;
auto state = encoding::initial_state();
```

```
do {
    std::string b = get_more_data();
    auto tv = make_text_view<encoding>(state, begin(b), end(b));
    auto it = begin(tv);
    while (it != end(tv))
        ...;
    state = it; // Trailing state is preserved in the end iterator. Save it
               // to seed state for the next loop iteration.
} while (!b.empty());
```

However, this leaves open the possibility for trailing code units at the end of an encoded text to go unnoticed. In a non-buffering scenario, an iterator might silently compare equal to the end iterator even though there are (possibly invalid) code units remaining.

Character Types

This library defines a character class template parameterized by character set type used to represent character values. The purpose of this class template is to make explicit the association of a code point value and a character set.

It has been suggested that `char32_t` be supported as a character type that is implicitly associated with the Unicode character set and that values of this type always be interpreted as Unicode code point values. This suggestion is intended to enable UTF-32 string literals to be directly usable as sequences of character values (in addition to being sequences of code unit and code point values). This has a cost in that it prohibits use of the `char32_t` type as a code unit or code point type for other encodings. Non-Unicode encodings, including the encodings used for ordinary and wide string literals, would still require a distinct character type (such as a specialization of the character class template) so that the correct character set can be inferred from objects of the character type.

This suggestion raises concerns for the author. To a certain degree, it can be accommodated by removing the current members of the character class template in favor of free functions and type trait templates. However, it results in ambiguities when enumerating the elements of a UTF-32 string literal; are the elements code point or character values? Well, the answer would be both (and code unit values as well). This raises the potential for inadvertently writing (generic) code that confuses code points and characters, runs as expected for UTF-32 encodings, but fails to compile for other encodings. The author would prefer to enforce correct code via the type system and is unaware of any particular benefits that the ability to treat UTF-32 string literals as sequences of character type would bring.

It has also been suggested that `char32_t` might suffice as the only character type; that decoding of any encoded string include implicit transcoding to Unicode code points. The author believes that this suggestion is not feasible for several reasons:

1. Some encodings use character sets that define characters such that round trip transcoding to Unicode and back fails to preserve the original code point value. For example, Shift-JIS (Microsoft code page 932) defines duplicate code points for the same character for compatibility with IBM and NEC character set extensions.
<https://support.microsoft.com/en-us/kb/170559>
2. Transcoding to Unicode for all non-Unicode encodings would carry non-negligible performance costs and would pessimize platforms such as IBM's z/OS that use EBCDIC by default for the non-Unicode execution character sets.

Locale Dependent Encodings

The ordinary and wide execution character sets are locale dependent; the interpretation of code point values that do not correspond to characters of the basic ordinary and wide execution character sets is determined at run-time based on locale settings. Yet, ordinary and wide string literals may contain universal-character-name designators that are transcoded at compile-time to some character set that is a superset of the corresponding basic character set and assumed to be a subset of the execution character set. These compile-time extended character sets are not currently named in the C++ standard.

Some compilers allow these compile-time extended character sets to be specified by command line options. For example, gcc supports `-fexec-charset=` and `-fwide-exec-charset=` options and Microsoft Visual C++ in Visual Studio 2015 Update 2 CTP recently added the `/execution-charset:` and `/utf-8` options. More information on these options can be found at:

- <https://gcc.gnu.org/onlinedocs/gcc-5.3.0/gcc/Preprocessor-Options.html#Preprocessor-Options>
- <https://blogs.msdn.microsoft.com/vcblog/2016/02/22/new-options-for-managing-character-sets-in-the-microsoft-cc-compiler/>

The `execution_character_encoding` and `execution_wide_character_encoding` type aliases defined by this library refer to encodings that use these unnamed character sets that are known at compile-time. This choice is motivated by future intentions to enable compile-time string manipulation and to allow avoiding the performance overhead of run-time locale awareness when an application is not locale dependent.

Though not currently specified, it may be appropriate to define additional encoding classes that implement locale awareness. It may also be more appropriate for the `execution_character_encoding` and `execution_wide_character_encoding` type aliases to refer to these locale dependent encodings and to introduce different names to refer to the extended compile-time execution encodings that are not currently named by the C++ standard.

Implementation Experience

A reference implementation of the described library is publicly available at https://github.com/tahonermann/text_view [\[Text view\]](#). The implementation requires a compiler that implements the C++ Concepts technical specification [\[Concepts\]](#). The only compilers known to do so at the time of this writing are gcc 6.2 and newer releases.

The reference implementation currently depends on Casey Carter and Eric Niebler's [cmcstl2](#) [\[cmcstl2\]](#) implementation of the ranges proposal [\[N4560\]](#) for concept definitions. The interfaces described in this document use the concept names from the ranges proposal [\[N4560\]](#), are intended to be used as specification, and should be considered authoritative. Any differences in behavior as defined by these definitions as compared to the reference implementation are unintentional and should be considered indicative of defects or limitations of the reference implementation and reported at https://github.com/tahonermann/text_view/issues.

Future Directions

Transcoding

Transcoding between encodings that use the same character set is currently possible. The following example transcodes a UTF-8 string to UTF-16.

```
std::string in = get_a_utf8_string();
std::u16string out;
std::back_inserter_iterator<std::u16string> out_it{out};
auto tv_in = make_text_view<utf8_encoding>(in);
auto tv_out = make_otext_iterator<utf16_encoding>(out_it);
std::copy(tv_in.begin(), tv_in.end(), tv_out);
```

Transcoding between encodings that use different character sets is not currently supported due to lack of interfaces to transcode a code point from one character set to the code point of a different one.

Additionally, naively transcoding between encodings using `std::copy()` works, but is not optimal; techniques are known to accelerate transcoding between some sets of encoding. For example, SIMD instructions can be utilized in some cases to transcode multiple code points in parallel.

Future work is intended to enable optimized transcoding and transcoding between distinct character sets.

Constexpr Support

Encodings that are not dependent on run-time support could conceivably support code point enumeration and transcoding to other encodings at compile time. This could be useful to conveniently provide text in alternative encodings at compile-time to meet requirements of external interfaces without incurring run-time overhead, having to write the string with hex escape sequences, or having to rely on preprocessing or other build time tools.

An example would be to provide a string in Modified UTF-8 for use in a JNI application.

```
auto tv = "Text with \0 embedded NUL"_modified_utf8;
// equivalent to:
auto tv = make_text_view<modified_utf8_encoding>({
    "Text with \xC0\x80 embedded NUL");
```

An additional example is that some of the proposals for reflections could benefit from the ability to transcode identifiers expressed in the basic source character encoding to a UTF-8 representation.

Unfortunately, user defined literals (UDLs) are currently unable to provide this support; though a constexpr UDL operator can be written, there is no known way to write the UDL such that an arbitrarily sized compile-time data structure can be returned, nor is there a way to instantiate a static buffer for the resulting transformation on a per string literal basis.

However, it is possible to perform string transformations at compile-time using a template constexpr function; so long as it is acceptable for the translated string to be embedded in another data structure.

```
template<int N>
struct my_str {
    char code_units[N];
};
```



```
template<int N>
constexpr my_str<N> make_my_str(const char (&str)[N]) {
    my_str<N> ms{};
    for (int i = 0; i < N; ++i) {
        char cu = str[i] ? str[i] + 1 : 0;
        ms.code_units[i] = cu;
    }
    return ms;
}

constexpr auto ms = make_my_str("text"); // ms.code_units[] == "ufyu"
```

One caveat of this approach is that the returned data structure owns the code unit sequence and is therefore more container-like than view-like.

Core language enhancements are probably necessary to make compile-time string literal translations a usable feature.

Unicode Normalization Iterators

Unicode [\[Unicode\]](#) encodings allow multiple code point sequences to denote the same character; this occurs with the use of combining characters. Unicode defines several normalization forms to enable consistent encoding of code point sequences.

Future work includes development of output iterators that perform Unicode normalization.

Unicode Grapheme Cluster Iterators

Unicode [\[Unicode\]](#) defines the concept of a grapheme cluster; a sequence of code points that includes nonspacing combining characters that, in general, should be processed as a unit.

Future work includes development of input iterators that enumerate grapheme clusters.

FAQ

Why do I have to specify the encoding for UTF-8 string literals?

This question refers to code like this:

```
auto tv = make_text_view<utf8_encoding>(u8"A UTF-8 string");
```

The argument to `make_text_view()` is a UTF-8 string literal. The compiler knows that it is a UTF-8 string. Yet, `make_text_view()` requires the encoding to be explicitly specified via a template argument. Why?

The answer is that ordinary and UTF-8 string literals have the same type; array of `const char`. The library is unable to implicitly determine that the provided string is UTF-8 encoded. At present, ranges that use `char` are assumed to be encoded per the `execution_character_encoding` (which may or may not be UTF-8).

A [proposal](#) [\[P0482R0\]](#) has been submitted to the EWG to add a `char8_t` type and to use it as the type for UTF-8 string and character literals (with appropriate accommodations for backward compatibility). If [P0482R0](#) [\[P0482R0\]](#) (or a future revision) were to be adopted, then it would be possible to assume (not infer) an encoding based on code unit type for all five of the encodings the standard states must be provided, and the requirement to explicitly name the encoding for calls to `make_text_view` with UTF-8 string literals could be lifted.

Can I define my own encodings? If so, How?

Yes. To do so, you'll need to define character set and encoding classes appropriate for your encoding.

```
class my_character_set {
public:
    using code_point_type = ...;
    static const char* get_name() noexcept;
};

struct my_encoding_state {};
struct my_encoding_state_transition {};

class my_encoding {
public:
    using state_type = my_encoding_state;
```

```

using state_transition_type = my_encoding_state_transition;
using character_type = character<my_character_set>;
using code_unit_type = ...;

static constexpr int min_code_units = ...;
static constexpr int max_code_units = ...;

static const state_type& initial_state() noexcept;

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode_state_transition(state_type &state,
                                                CUIT &out,
                                                const state_transition_type &stt,
                                                int &encoded_code_units)

    noexcept(...);

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode(state_type &state,
                                CUIT &out,
                                character_type c,
                                int &encoded_code_units)

    noexcept(...);

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status decode(state_type &state,
                                CUIT &in_next,
                                CUST in_end,
                                character_type &c,
                                int &decoded_code_units)

    noexcept(...);

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status rdecode(state_type &state,
                                CUIT &in_next,
                                CUST in_end,
                                character_type &c,
                                int &decoded_code_units)

    noexcept(...);
};

```

Technical Specifications

Header <experimental/text_view> synopsis

```

namespace std {
namespace experimental {
inline namespace text {

// concepts:
template<typename T> concept bool CodeUnit();
template<typename T> concept bool CodePoint();
template<typename T> concept bool CharacterSet();
template<typename T> concept bool Character();
template<typename T> concept bool CodeUnitIterator();
template<typename T, typename V> concept bool CodeUnitOutputIterator();
template<typename T> concept bool TextEncodingState();
template<typename T> concept bool TextEncodingStateTransition();
template<typename T> concept bool TextErrorPolicy();
template<typename T> concept bool TextEncoding();
template<typename T, typename I> concept bool TextEncoder();
template<typename T, typename I> concept bool TextForwardDecoder();
template<typename T, typename I> concept bool TextBidirectionalDecoder();
template<typename T, typename I> concept bool TextRandomAccessDecoder();
template<typename T> concept bool TextIterator();
template<typename T> concept bool TextOutputIterator();
template<typename T> concept bool TextInputIterator();
template<typename T> concept bool TextForwardIterator();
template<typename T> concept bool TextBidirectionalIterator();
template<typename T> concept bool TextRandomAccessIterator();
template<typename T, typename I> concept bool TextSentinel();
template<typename T> concept bool TextView();
template<typename T> concept bool TextInputView();

```

```

template<typename T> concept bool TextForwardView();
template<typename T> concept bool TextBidirectionalView();
template<typename T> concept bool TextRandomAccessView();

// error policies:
class text_error_policy;
class text_strict_error_policy;
class text_permissive_error_policy;
using text_default_error_policy = text_strict_error_policy;

// error handling:
enum class encode_status : int {
    no_error = /* implementation-defined */,
    invalid_character = /* implementation-defined */,
    invalid_state_transition = /* implementation-defined */
};
enum class decode_status : int {
    no_error = /* implementation-defined */,
    no_character = /* implementation-defined */,
    invalid_code_unit_sequence = /* implementation-defined */,
    underflow = /* implementation-defined */
};
constexpr inline bool status_ok(encode_status es) noexcept;
constexpr inline bool status_ok(decode_status ds) noexcept;
constexpr inline bool error_occurred(encode_status es) noexcept;
constexpr inline bool error_occurred(decode_status ds) noexcept;
const char* status_message(encode_status es) noexcept;
const char* status_message(decode_status ds) noexcept;

// exception classes:
class text_error;
class text_encode_error;
class text_decode_error;

// character sets:
class any_character_set;
class basic_execution_character_set;
class basic_execution_wide_character_set;
class unicode_character_set;

// implementation defined character set type aliases:
using execution_character_set = /* implementation-defined */ ;
using execution_wide_character_set = /* implementation-defined */ ;
using universal_character_set = /* implementation-defined */ ;

// character set identification:
class character_set_id;

template<CharacterSet CST>
    inline character_set_id get_character_set_id();

// character set information:
class character_set_info;

template<CharacterSet CST>
    inline const character_set_info& get_character_set_info();
const character_set_info& get_character_set_info(character_set_id id);

// character set and encoding traits:
template<typename T>
    using code_unit_type_t = /* implementation-defined */ ;
template<typename T>
    using code_point_type_t = /* implementation-defined */ ;
template<typename T>
    using character_set_type_t = /* implementation-defined */ ;
template<typename T>
    using character_type_t = /* implementation-defined */ ;
template<typename T>
    using encoding_type_t = /* implementation-defined */ ;
template<typename T>
    using default_encoding_type_t = /* implementation-defined */ ;

// characters:
template<CharacterSet CST> class character;
template<> class character<any_character_set>;

template<CharacterSet CST>
    bool operator==(const character<any_character_set> &lhs,
                    const character<CST> &rhs);
template<CharacterSet CST>
    bool operator==(const character<CST> &lhs,
                    const character<any_character_set> &rhs);
template<CharacterSet CST>

```

```

    bool operator!=(const character<any_character_set> &lhs,
                    const character<CST> &rhs);
template<CharacterSet CST>
    bool operator!=(const character<CST> &lhs,
                    const character<any_character_set> &rhs);

// encoding state and transition types:
class trivial_encoding_state;
class trivial_encoding_state_transition;
class utf8bom_encoding_state;
class utf8bom_encoding_state_transition;
class utf16bom_encoding_state;
class utf16bom_encoding_state_transition;
class utf32bom_encoding_state;
class utf32bom_encoding_state_transition;

// encodings:
class basic_execution_character_encoding;
class basic_execution_wide_character_encoding;
#if defined(__STDC_ISO_10646__)
class iso_10646_wide_character_encoding;
#endif // __STDC_ISO_10646__
class utf8_encoding;
class utf8bom_encoding;
class utf16_encoding;
class utf16be_encoding;
class utf16le_encoding;
class utf16bom_encoding;
class utf32_encoding;
class utf32be_encoding;
class utf32le_encoding;
class utf32bom_encoding;

// implementation defined encoding type aliases:
using execution_character_encoding = /* implementation-defined */ ;
using execution_wide_character_encoding = /* implementation-defined */ ;
using char8_character_encoding = /* implementation-defined */ ;
using char16_character_encoding = /* implementation-defined */ ;
using char32_character_encoding = /* implementation-defined */ ;

// itext_iterator:
template<TextEncoding ET,
        ranges::View VT,
        TextErrorPolicy TEP = text_default_error_policy>
    requires TextForwardDecoder<ET, /* implementation-defined */ >()
    class itext_iterator;

// itext_sentinel:
template<TextEncoding ET,
        ranges::View VT,
        TextErrorPolicy TEP = text_default_error_policy>
    class itext_sentinel;

// otext_iterator:
template<TextEncoding ET,
        CodeUnitOutputIterator<code_unit_type_t<ET>> CUIT,
        TextErrorPolicy TEP = text_default_error_policy>
    class otext_iterator;

// otext_iterator factory functions:
template<TextEncoding ET,
        TextErrorPolicy TEP,
        CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
    auto make_otext_iterator(typename ET::state_type state, IT out)
    -> otext_iterator<ET, IT>;
template<TextEncoding ET,
        CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
    auto make_otext_iterator(typename ET::state_type state, IT out)
    -> otext_iterator<ET, IT>;
template<TextEncoding ET,
        TextErrorPolicy TEP,
        CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
    auto make_otext_iterator(IT out)
    -> otext_iterator<ET, IT>;
template<TextEncoding ET,
        CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
    auto make_otext_iterator(IT out)
    -> otext_iterator<ET, IT>;

// basic_text_view:
template<TextEncoding ET,
        ranges::View VT,
        TextErrorPolicy TEP = text_default_error_policy>

```

```

class basic_text_view;

// basic_text_view type aliases:
using text_view = basic_text_view<execution_character_encoding,
                                   /* implementation-defined */ >;
using wtext_view = basic_text_view<execution_wide_character_encoding,
                                   /* implementation-defined */ >;
using u8text_view = basic_text_view<char8_character_encoding,
                                   /* implementation-defined */ >;
using u16text_view = basic_text_view<char16_character_encoding,
                                   /* implementation-defined */ >;
using u32text_view = basic_text_view<char32_character_encoding,
                                   /* implementation-defined */ >;

// basic_text_view factory functions:
template<TextEncoding ET, ranges::InputIterator IT, ranges::Sentinel<IT> ST>
    auto make_text_view(typename ET::state_type state, IT first, ST last)
    -> basic_text_view<ET, /* implementation-defined */ >;
template<ranges::InputIterator IT, ranges::Sentinel<IT> ST>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<IT>>;
    }
    auto make_text_view(typename default_encoding_type_t<ranges::value_type_t<IT>>::state_type state,
                        IT first,
                        ST last)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;
template<TextEncoding ET, ranges::InputIterator IT, ranges::Sentinel<IT> ST>
    auto make_text_view(IT first, ST last)
    -> basic_text_view<ET, /* implementation-defined */ >;
template<ranges::InputIterator IT, ranges::Sentinel<IT> ST>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<IT>>;
    }
    auto make_text_view(IT first, ST last)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;
template<TextEncoding ET, ranges::ForwardIterator IT>
    auto make_text_view(typename ET::state_type state,
                        IT first,
                        typename std::make_unsigned<ranges::difference_type_t<IT>>::type n)
    -> basic_text_view<ET, /* implementation-defined */ >;
template<ranges::ForwardIterator IT>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<IT>>;
    }
    auto make_text_view(typename default_encoding_type_t<ranges::value_type_t<IT>>::state_type state,
                        IT first,
                        typename std::make_unsigned<ranges::difference_type_t<IT>>::type n)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;
template<TextEncoding ET, ranges::ForwardIterator IT>
    auto make_text_view(IT first,
                        typename std::make_unsigned<ranges::difference_type_t<IT>>::type n)
    -> basic_text_view<ET, /* implementation-defined */ >;
template<ranges::ForwardIterator IT>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<IT>>;
    }
    auto make_text_view(IT first,
                        typename std::make_unsigned<ranges::difference_type_t<IT>>::type n)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;
template<TextEncoding ET, ranges::InputRange Iterable>
    auto make_text_view(typename ET::state_type state,
                        const Iterable &iterable)
    -> basic_text_view<ET, /* implementation-defined */ >;
template<ranges::InputRange Iterable>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>;
    }
    auto make_text_view(typename default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>::state_type state,
                        const Iterable &iterable)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>, /* implementation-defined */ >;
template<TextEncoding ET, ranges::InputRange Iterable>
    auto make_text_view(const Iterable &iterable)
    -> basic_text_view<ET, /* implementation-defined */ >;
template<ranges::InputRange Iterable>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>;
    }
    auto make_text_view(const Iterable &iterable)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>, /* implementation-defined */ >;
template<TextInputIterator TIT, TextSentinel<TIT> TST>
    auto make_text_view(TIT first, TST last)
    -> basic_text_view<ET, /* implementation-defined */ >;
template<TextView TVT>

```

```
TVT make_text_view(TVT tv);

} // inline namespace text
} // namespace experimental
} // namespace std
```

Concepts

- [Concept CodeUnit](#)
- [Concept CodePoint](#)
- [Concept CharacterSet](#)
- [Concept Character](#)
- [Concept CodeUnitIterator](#)
- [Concept CodeUnitOutputIterator](#)
- [Concept TextEncodingState](#)
- [Concept TextEncodingStateTransition](#)
- [Concept TextErrorPolicy](#)
- [Concept TextEncoding](#)
- [Concept TextEncoder](#)
- [Concept TextForwardDecoder](#)
- [Concept TextBidirectionalDecoder](#)
- [Concept TextRandomAccessDecoder](#)
- [Concept TextIterator](#)
- [Concept TextSentinel](#)
- [Concept TextOutputIterator](#)
- [Concept TextInputIterator](#)
- [Concept TextForwardIterator](#)
- [Concept TextBidirectionalIterator](#)
- [Concept TextRandomAccessIterator](#)
- [Concept TextView](#)
- [Concept TextInputView](#)
- [Concept TextForwardView](#)
- [Concept TextBidirectionalView](#)
- [Concept TextRandomAccessView](#)

Concept CodeUnit

The CodeUnit concept specifies requirements for a type usable as the code unit type of a string type.

CodeUnit<T>() is satisfied if and only if:

- `std::is_integral<T>::value` is true
- and at least one of:
 - `std::is_unsigned<T>::value` is true.
 - `std::is_same<std::remove_cv_t<T>, char>::value` is true.
 - `std::is_same<std::remove_cv_t<T>, wchar_t>::value` is true.

```
template<typename T> concept bool CodeUnit() {
    return /* implementation-defined */ ;
}
```

Concept CodePoint

The CodePoint concept specifies requirements for a type usable as the code point type of a character set type.

CodePoint<T>() is satisfied if and only if:

- `std::is_integral<T>::value` is true
- and at least one of:
 - `std::is_unsigned<T>::value` is true.
 - `std::is_same<std::remove_cv_t<T>, char>::value` is true.
 - `std::is_same<std::remove_cv_t<T>, wchar_t>::value` is true.

```
template<typename T> concept bool CodePoint() {
    return /* implementation-defined */ ;
}
```

Concept CharacterSet

The `CharacterSet` concept specifies requirements for a type that describes a character set. Such a type has a member typedef-name declaration for a type that satisfies `CodePoint`, a static member function that returns a name for the character set, and a static member function that returns a code point value to be used to construct a substitution character to stand in when errors occur during encoding and decoding operations when the permissive error policy is in effect.

```
template<typename T> concept bool CharacterSet() {
    return CodePoint<code_point_type_t<T>>()
        && requires () {
            { T::get_name() } noexcept -> const char *;
            { T::get_substitution_code_point() } noexcept -> code_point_type_t<T>;
        };
}
```

Concept Character

The `Character` concept specifies requirements for a type that describes a character as defined by an associated character set. Non-static member functions provide access to the code point value of the described character. Types that satisfy `Character` are regular and copyable.

```
template<typename T> concept bool Character() {
    return ranges::Regular<T>()
        && ranges::Constructible<T, code_point_type_t<character_set_type_t<T>>>()
        && CharacterSet<character_set_type_t<T>>()
        && requires (T t,
                    const T ct,
                    code_point_type_t<character_set_type_t<T>> cp)
        {
            { t.set_code_point(cp) } noexcept;
            { ct.get_code_point() } noexcept
                -> code_point_type_t<character_set_type_t<T>>;
            { ct.get_character_set_id() }
                -> character_set_id;
        };
}
```

Concept CodeUnitIterator

The `CodeUnitIterator` concept specifies requirements of an iterator that has a value type that satisfies `CodeUnit`.

```
template<typename T> concept bool CodeUnitIterator() {
    return ranges::Iterator<T>()
        && CodeUnit<ranges::value_type_t<T>>();
}
```

Concept CodeUnitOutputIterator

The `CodeUnitOutputIterator` concept specifies requirements of an output iterator that can be assigned from a type that satisfies `CodeUnit`.

```
template<typename T, typename V> concept bool CodeUnitOutputIterator() {
    return ranges::OutputIterator<T, V>()
        && CodeUnit<V>();
}
```

Concept TextEncodingState

The `TextEncodingState` concept specifies requirements of types that hold encoding state. Such types are semiregular.

```
template<typename T> concept bool TextEncodingState() {
    return ranges::Semiregular<T>();
}
```


Concept TextEncodingStateTransition

The `TextEncodingStateTransition` concept specifies requirements of types that hold encoding state transitions. Such types are semiregular.

```
template<typename T> concept bool TextEncodingStateTransition() {
    return ranges::Semiregular<T>();
}
```

Concept TextErrorPolicy

The `TextErrorPolicy` concept specifies requirements of types used to specify error handling policies. Such types are semiregular class types that derive from class `text_error_policy`.

```
template<typename T> concept bool TextErrorPolicy() {
    return ranges::Semiregular<T>()
        && ranges::DerivedFrom<T, text_error_policy>
        && !ranges::Same<std::remove_cv_t<T>, text_error_policy>();
}
```

Concept TextEncoding

The `TextEncoding` concept specifies requirements of types that define an encoding. Such types define member types that identify the code unit, character, encoding state, and encoding state transition types, a static member function that returns an initial encoding state object that defines the encoding state at the beginning of a sequence of encoded characters, and static data members that specify the minimum and maximum number of code units used to encode any single character.

```
template<typename T> concept bool TextEncoding() {
    return requires () {
        { T::min_code_units } noexcept -> int;
        { T::max_code_units } noexcept -> int;
    }
    && TextEncodingState<typename T::state_type>()
    && TextEncodingStateTransition<typename T::state_transition_type>()
    && CodeUnit<code_unit_type_t<T>>()
    && Character<character_type_t<T>>()
    && requires () {
        { T::initial_state() } noexcept
            -> const typename T::state_type&;
    };
};
```

Concept TextEncoder

The `TextEncoder` concept specifies requirements of types that are used to encode characters using a particular code unit iterator that satisfies `OutputIterator`. Such a type satisfies `TextEncoding` and defines static member functions used to encode state transitions and characters.

```
template<typename T, typename I> concept bool TextEncoder() {
    return TextEncoding<T>()
        && ranges::OutputIterator<CUIT, code_unit_type_t<T>>()
        && requires (
            typename T::state_type &state,
            CUIT &out,
            typename T::state_transition_type stt,
            int &encoded_code_units)
        {
            { T::encode_state_transition(state, out, stt, encoded_code_units) }
                -> encode_status;
        }
        && requires (
            typename T::state_type &state,
            CUIT &out,
            character_type_t<T> c,
            int &encoded_code_units)
        {
            { T::encode(state, out, c, encoded_code_units) }
                -> encode_status;
        };
};
```

```
}
```

Concept TextForwardDecoder

The `TextForwardDecoder` concept specifies requirements of types that are used to decode characters using a particular code unit iterator that satisfies `ForwardIterator`. Such a type satisfies `TextEncoding` and defines a static member function used to decode state transitions and characters.

```
template<typename T, typename I> concept bool TextForwardDecoder() {
    return TextEncoding<T>()
        && ranges::ForwardIterator<CUIT>()
        && ranges::ConvertibleTo<ranges::value_type_t<CUIT>,
                                code_unit_type_t<T>>()

        && requires (
            typename T::state_type &state,
            CUIT &in_next,
            CUIT in_end,
            character_type_t<T> &c,
            int &decoded_code_units)
        {
            { T::decode(state, in_next, in_end, c, decoded_code_units) }
            -> decode_status;
        };
}
```

Concept TextBidirectionalDecoder

The `TextBidirectionalDecoder` concept specifies requirements of types that are used to decode characters using a particular code unit iterator that satisfies `BidirectionalIterator`. Such a type also satisfies `TextForwardDecoder` and defines a static member function used to decode state transitions and characters in the reverse order of their encoding.

```
template<typename T, typename I> concept bool TextBidirectionalDecoder() {
    return TextForwardDecoder<T, CUIT>()
        && ranges::BidirectionalIterator<CUIT>()
        && requires (
            typename T::state_type &state,
            CUIT &in_next,
            CUIT in_end,
            character_type_t<T> &c,
            int &decoded_code_units)
        {
            { T::rdecode(state, in_next, in_end, c, decoded_code_units) }
            -> decode_status;
        };
}
```

Concept TextRandomAccessDecoder

The `TextRandomAccessDecoder` concept specifies requirements of types that are used to decode characters using a particular code unit iterator that satisfies `RandomAccessIterator`. Such a type also satisfies `TextBidirectionalDecoder`, requires that the minimum and maximum number of code units used to encode any character have the same value, and that the encoding state be an empty type.

```
template<typename T, typename I> concept bool TextRandomAccessDecoder() {
    return TextBidirectionalDecoder<T, CUIT>()
        && ranges::RandomAccessIterator<CUIT>()
        && T::min_code_units == T::max_code_units
        && std::is_empty<typename T::state_type>::value;
}
```

Concept TextIterator

The `TextIterator` concept specifies requirements of iterator types that are used to encode and decode characters as an encoded sequence of code units. Encoding state and error indication is held in each iterator instance and is made accessible via non-static member functions.

```
template<typename T> concept bool TextIterator() {
    return ranges::Iterator<T>()
```

```

    && TextEncoding<encoding_type_t<T>>>()
    && TextErrorPolicy<typename T::error_policy>()
    && TextEncodingState<typename T::state_type>()
    && requires (const T ct) {
        { ct.state() } noexcept
        -> const typename encoding_type_t<T::state_type&;
        { ct.error_occurred() } noexcept
        -> bool;
    };
};
}

```

Concept TextSentinel

The `TextSentinel` concept specifies requirements of types that are used to mark the end of a range of encoded characters. A type `T` that satisfies `TextIterator` also satisfies `TextSentinel<T>` there by enabling `TextIterator` types to be used as sentinels.

```

template<typename T, typename I> concept bool TextSentinel() {
    return ranges::Sentinel<T, I>()
        && TextIterator<I>()
        && TextErrorPolicy<typename T::error_policy>();
}

```

Concept TextOutputIterator

The `TextOutputIterator` concept refines `TextIterator` with a requirement that the type also satisfy `ranges::OutputIterator` for the character type of the associated encoding and that a member function be provided for retrieving error information.

```

template<typename T> concept bool TextOutputIterator() {
    return TextIterator<T>()
        && ranges::OutputIterator<T, character_type_t<encoding_type_t<T>>>()
        && requires (const T ct) {
            { ct.get_error() } noexcept
            -> encode_status;
        };
}

```

Concept TextInputIterator

The `TextInputIterator` concept refines `TextIterator` with requirements that the type also satisfy `ranges::InputIterator`, that the iterator value type satisfy `Character`, and that a member function be provided for retrieving error information.

```

template<typename T> concept bool TextInputIterator() {
    return TextIterator<T>()
        && ranges::InputIterator<T>()
        && Character<ranges::value_type_t<T>>()
        && requires (const T ct) {
            { ct.get_error() } noexcept
            -> decode_status;
        };
}

```

Concept TextForwardIterator

The `TextForwardIterator` concept refines `TextInputIterator` with a requirement that the type also satisfy `ranges::ForwardIterator`.

```

template<typename T> concept bool TextForwardIterator() {
    return TextInputIterator<T>()
        && ranges::ForwardIterator<T>();
}

```

Concept TextBidirectionalIterator

The `TextBidirectionalIterator` concept refines `TextForwardIterator` with a requirement that the type also satisfy `ranges::BidirectionalIterator`.

```

template<typename T> concept bool TextBidirectionalIterator() {

```

```

return TextForwardIterator<T>()
    && ranges::BidirectionalIterator<T>();
}

```

Concept TextRandomAccessIterator

The `TextRandomAccessIterator` concept refines `TextBidirectionalIterator` with a requirement that the type also satisfy `ranges::RandomAccessIterator`.

```

template<typename T> concept bool TextRandomAccessIterator() {
    return TextBidirectionalIterator<T>()
        && ranges::RandomAccessIterator<T>();
}

```

Concept TextView

The `TextView` concept specifies requirements of types that provide view access to an underlying code unit range. Such types satisfy `ranges::View`, provide iterators that satisfy `TextIterator`, define member types that identify the encoding, encoding state, and underlying code unit range and iterator types. Non-static member functions are provided to access the underlying code unit range and initial encoding state.

Types that satisfy `TextView` do not own the underlying code unit range and are copyable in constant time. The lifetime of the underlying range must exceed the lifetime of referencing `TextView` objects.

```

template<typename T> concept bool TextView() {
    return ranges::View<T>()
        && TextIterator<ranges::iterator_t<T>>()
        && TextEncoding<encoding_type_t<T>>()
        && ranges::View<typename T::view_type>()
        && TextErrorPolicy<typename T::error_policy>()
        && TextEncodingState<typename T::state_type>()
        && CodeUnitIterator<code_unit_iterator_t<T>>()
        && requires (T t, const T ct) {
            { ct.base() } noexcept
                -> const typename T::view_type&;
            { ct.initial_state() } noexcept
                -> const typename T::state_type&;
        };
}

```

Concept TextInputView

The `TextInputView` concept refines `TextView` with a requirement that the view's iterator type also satisfy `TextInputIterator`.

```

template<typename T> concept bool TextInputView() {
    return TextView<T>()
        && TextInputIterator<ranges::iterator_t<T>>();
}

```

Concept TextForwardView

The `TextForwardView` concept refines `TextInputView` with a requirement that the view's iterator type also satisfy `TextForwardIterator`.

```

template<typename T> concept bool TextForwardView() {
    return TextInputView<T>()
        && TextForwardIterator<ranges::iterator_t<T>>();
}

```

Concept TextBidirectionalView

The `TextBidirectionalView` concept refines `TextForwardView` with a requirement that the view's iterator type also satisfy `TextBidirectionalIterator`.

```

template<typename T> concept bool TextBidirectionalView() {
    return TextForwardView<T>()

```

```
&& TextBidirectionalIterator<ranges::iterator_t<T>>>());  
}
```

Concept TextRandomAccessView

The `TextRandomAccessView` concept refines `TextBidirectionalView` with a requirement that the view's iterator type also satisfy `TextRandomAccessIterator`.

```
template<typename T> concept bool TextRandomAccessView() {  
    return TextBidirectionalView<T>()  
        && TextRandomAccessIterator<ranges::iterator_t<T>>>();  
}
```

Error Policies

- [Class text_error_policy](#)
- [Class text_strict_error_policy](#)
- [Class text_permissive_error_policy](#)
- [Alias text_default_error_policy](#)

Class text_error_policy

Class `text_error_policy` is a base class from which all text error policy classes must derive.

```
class text_error_policy {};
```

Class text_strict_error_policy

The `text_strict_error_policy` class is a policy class that specifies that exceptions be thrown for errors that occur during encoding and decoding operations initiated through text iterators. This class satisfies `TextErrorPolicy`.

```
class text_strict_error_policy : public text_error_policy {};
```

Class text_permissive_error_policy

The `class_text_permissive_error_policy` class is a policy class that specifies that substitution characters such as the Unicode replacement character `U+FFFD` be substituted in place of errors that occur during encoding and decoding operations initiated through text iterators. This class satisfies `TextErrorPolicy`.

```
class text_permissive_error_policy : public text_error_policy {};
```

Alias text_default_error_policy

The `text_default_error_policy` alias specifies the default text error policy. Conforming implementations must alias this to `text_strict_error_policy`, but may have options to select an alternative default policy for environments that do not support exceptions. The referred class shall satisfy `TextErrorPolicy`.

```
using text_default_error_policy = text_strict_error_policy;
```

Error Status

- [Enum encode_status](#)
- [Enum decode_status](#)
- [status_ok](#)
- [error_occurred](#)
- [status_message](#)

Enum encode_status

The `encode_status` enumeration type defines enumerators used to report errors that occur during text encoding operations.

The `no_error` enumerator indicates that no error has occurred.

The `invalid_character` enumerator indicates that an attempt was made to encode a character that was not valid for the encoding.

The `invalid_state_transition` enumerator indicates that an attempt was made to encode a state transition that was not valid for the encoding.

```
enum class encode_status : int {
    no_error = /* implementation-defined */,
    invalid_character = /* implementation-defined */,
    invalid_state_transition = /* implementation-defined */
};
```

Enum `decode_status`

The `decode_status` enumeration type defines enumerators used to report errors that occur during text decoding operations.

The `no_error` enumerator indicates that no error has occurred.

The `no_character` enumerator indicates that no error has occurred, but that no character was decoded for a code unit sequence. This typically indicates that the code unit sequence represents an encoding state transition such as for an escape sequence or byte order marker.

The `invalid_code_unit_sequence` enumerator indicates that an attempt was made to decode an invalid code unit sequence.

The `underflow` enumerator indicates that the end of the input range was encountered before a complete code unit sequence was decoded.

```
enum class decode_status : int {
    no_error = /* implementation-defined */,
    no_character = /* implementation-defined */,
    invalid_code_unit_sequence = /* implementation-defined */,
    underflow = /* implementation-defined */
};
```

`status_ok`

The `status_ok` function returns `true` if the `encode_status` argument value is `encode_status::no_error` or if the `decode_status` argument is either of `decode_status::no_error` or `decode_status::no_character`. `false` is returned for all other values.

```
constexpr inline bool status_ok(encode_status es) noexcept;
constexpr inline bool status_ok(decode_status ds) noexcept;
```

`error_occurred`

The `error_occurred` function returns `false` if the `encode_status` argument value is `encode_status::no_error` or if the `decode_status` argument is either of `decode_status::no_error` or `decode_status::no_character`. `true` is returned for all other values.

```
constexpr inline bool error_occurred(encode_status es) noexcept;
constexpr inline bool error_occurred(decode_status ds) noexcept;
```

`status_message`

The `status_message` function returns a pointer to a statically allocated string containing a short description of the value of the `encode_status` or `decode_status` argument.

```
const char* status_message(encode_status es) noexcept;
const char* status_message(decode_status ds) noexcept;
```

Exceptions

- [Class text_error](#)
- [Class text_encode_error](#)
- [Class text_decode_error](#)

Class text_error

The `text_error` class defines the base class for the types of objects thrown as exceptions to report errors detected during text processing.

```
class text_error : public std::runtime_error
{
public:
    using std::runtime_error::runtime_error;
};
```

Class text_encode_error

The `text_encode_error` class defines the types of objects thrown as exceptions to report errors detected during encoding of a character. Objects of this type are generally thrown in response to an attempt to encode a character with an invalid code point value, or to encode an invalid state transition.

```
class text_encode_error : public text_error
{
public:
    explicit text_encode_error(encode_status es) noexcept;

    const encode_status& status_code() const noexcept;

private:
    encode_status es; // exposition only
};
```

Class text_decode_error

The `text_decode_error` class defines the types of objects thrown as exceptions to report errors detected during decoding of a code unit sequence. Objects of this type are generally thrown in response to an attempt to decode an ill-formed code unit sequence, a code unit sequence that specifies an invalid code point value, or a code unit sequence that specifies an invalid state transition.

```
class text_decode_error : public text_error
{
public:
    explicit text_decode_error(decode_status ds) noexcept;

    const decode_status& status_code() const noexcept;

private:
    decode_status ds; // exposition only
};
```

Type Traits

- [code_unit_type_t](#)
- [code_point_type_t](#)
- [character_set_type_t](#)
- [character_type_t](#)
- [encoding_type_t](#)
- [default_encoding_type_t](#)

code_unit_type_t

The `code_unit_type_t` type alias template provides convenient means for selecting the associated code unit type of some other type, such as an encoding type that satisfies `TextEncoding`. The aliased type is the same as `typename T::code_unit_type`.

```
template<typename T>
    using code_unit_type_t = /* implementation-defined */ ;
```

code_point_type_t

The `code_point_type_t` type alias template provides convenient means for selecting the associated code point type of some other type, such as a type that satisfies `CharacterSet` or `Character`. The aliased type is the same as `typename T::code_point_type`.

```
template<typename T>
using code_point_type_t = /* implementation-defined */ ;
```

character_set_type_t

The `character_set_type_t` type alias template provides convenient means for selecting the associated character set type of some other type, such as a type that satisfies `Character`. The aliased type is the same as `typename T::character_set_type`.

```
template<typename T>
using character_set_type_t = /* implementation-defined */ ;
```

character_type_t

The `character_type_t` type alias template provides convenient means for selecting the associated character type of some other type, such as a type that satisfies `TextEncoding`. The aliased type is the same as `typename T::character_type`.

```
template<typename T>
using character_type_t = /* implementation-defined */ ;
```

encoding_type_t

The `encoding_type_t` type alias template provides convenient means for selecting the associated encoding type of some other type, such as a type that satisfies `TextIterator` or `TextView`. The aliased type is the same as `typename T::encoding_type`.

```
template<typename T>
using encoding_type_t = /* implementation-defined */ ;
```

default_encoding_type_t

The `default_encoding_type_t` type alias template resolves to the default encoding type, if any, for a given type, such as a type that satisfies `CodeUnit`. Specializations are provided for the following cv-unqualified and reference removed fundamental types. Otherwise, the alias will attempt to resolve against a `default_encoding_type` member type.

When <code>std::remove_cv_t<std::remove_reference_t<T>></code> is ...	the default encoding is ...
<code>char</code>	<code>execution_character_encoding</code>
<code>wchar_t</code>	<code>execution_wide_character_encoding</code>
<code>char16_t</code>	<code>char16_character_encoding</code>
<code>char32_t</code>	<code>char32_character_encoding</code>

```
template<typename T>
using default_encoding_type_t = /* implementation-defined */ ;
```

Character Sets

- [Class any character set](#)
- [Class basic execution character set](#)
- [Class basic execution wide character set](#)
- [Class unicode character set](#)
- [Character set type aliases](#)

Class any_character_set

The `any_character_set` class provides a generic character set type used when a specific character set type is unknown or when the ability

to switch between specific character sets is required. This class satisfies the `CharacterSet` concept and has an implementation defined `code_point_type` that is able to represent code point values from all of the implementation provided character set types. The code point returned by `get_substitution_code_point` is implementation defined.

```
class any_character_set {
public:
    using code_point_type = /* implementation-defined */;

    static const char* get_name() noexcept {
        return "any_character_set";
    }

    static constexpr code_point_type get_substitution_code_point() noexcept;
};
```

Class `basic_execution_character_set`

The `basic_execution_character_set` class represents the basic execution character set specified in [lex.charset]p3 of the C++ standard. This class satisfies the `CharacterSet` concept and has a `code_point_type` member type that aliases `char`. The code point returned by `get_substitution_code_point` is the code point for the `'?'` character.

```
class basic_execution_character_set {
public:
    using code_point_type = char;

    static const char* get_name() noexcept {
        return "basic_execution_character_set";
    }

    static constexpr code_point_type get_substitution_code_point() noexcept;
};
```

Class `basic_execution_wide_character_set`

The `basic_execution_wide_character_set` class represents the basic execution wide character set specified in [lex.charset]p3 of the C++ standard. This class satisfies the `CharacterSet` concept and has a `code_point_type` member type that aliases `wchar_t`. The code point returned by `get_substitution_code_point` is the code point for the `L'?'` character.

```
class basic_execution_wide_character_set {
public:
    using code_point_type = wchar_t;

    static const char* get_name() noexcept {
        return "basic_execution_wide_character_set";
    }

    static constexpr code_point_type get_substitution_code_point() noexcept;
};
```

Class `unicode_character_set`

The `unicode_character_set` class represents the Unicode character set. This class satisfies the `CharacterSet` concept and has a `code_point_type` member type that aliases `char32_t`. The code point returned by `get_substitution_code_point` is the U+FFFD Unicode replacement character.

```
class unicode_character_set {
public:
    using code_point_type = char32_t;

    static const char* get_name() noexcept {
        return "unicode_character_set";
    }

    static constexpr code_point_type get_substitution_code_point() noexcept;
};
```

Character set type aliases

The `execution_character_set`, `execution_wide_character_set`, and `universal_character_set` type aliases reflect the implementation defined execution, wide execution, and universal character sets specified in [lex.charset]p2-3 of the C++ standard.

The character set aliased by `execution_character_set` must be a superset of the `basic_execution_character_set` character set. This alias refers to the character set that the compiler assumes during translation; the character set that the compiler uses when translating characters specified by universal-character-name designators in ordinary string literals, not the locale sensitive run-time execution character set.

The character set aliased by `execution_wide_character_set` must be a superset of the `basic_execution_wide_character_set` character set. This alias refers to the character set that the compiler assumes during translation; the character set that the compiler uses when translating characters specified by universal-character-name designators in wide string literals, not the locale sensitive run-time execution wide character set.

The character set aliased by `universal_character_set` must be a superset of the `unicode_character_set` character set.

```
using execution_character_set = /* implementation-defined */ ;
using execution_wide_character_set = /* implementation-defined */ ;
using universal_character_set = /* implementation-defined */ ;
```

Character Set Identification

- [Class `character\_set\_id`](#)
- [get `character\_set\_id`](#)

Class `character_set_id`

The `character_set_id` class provides unique, opaque values used to identify character sets at run-time. Values of this type are produced by `get_character_set_id()` and can be passed to `get_character_set_info()` to obtain character set information. Values of this type are copy constructible, copy assignable, equality comparable, and strictly totally ordered.

```
class character_set_id {
public:
    character_set_id() = delete;

    friend bool operator==(character_set_id lhs, character_set_id rhs) noexcept;
    friend bool operator!=(character_set_id lhs, character_set_id rhs) noexcept;

    friend bool operator<(character_set_id lhs, character_set_id rhs) noexcept;
    friend bool operator>(character_set_id lhs, character_set_id rhs) noexcept;
    friend bool operator<=(character_set_id lhs, character_set_id rhs) noexcept;
    friend bool operator>=(character_set_id lhs, character_set_id rhs) noexcept;
};
```

get `character_set_id`

`get_character_set_id()` returns a unique, opaque value for the chracter set type specified by the template parameter.

```
template<CharacterSet CST>
    inline character_set_id get_character_set_id();
```

Character Set Information

- [Class `character\_set\_info`](#)
- [get `character\_set\_info`](#)

Class `character_set_info`

The `character_set_info` class stores information about a character set. Values of this type are produced by the `get_character_set_info()` functions based on a character set type or ID.

```
class character_set_info {
public:
    character_set_info() = delete;

    character_set_id get_id() const noexcept;
```

```

    const char* get_name() const noexcept;

private:
    character_set_id id; // exposition only
};

```

get_character_set_info

The `get_character_set_info()` functions return a reference to a `character_set_info` object based on a character set type or ID.

```

const character_set_info& get_character_set_info(character_set_id id);

template<CharacterSet CST>
    inline const character_set_info& get_character_set_info();

```

Characters

- [Class template character](#)

Class template character

Objects of `character` class template specialization type define a character via the association of a code point value and a character set. The specialization provided for the `any_character_set` type is used to maintain a dynamic character set association while specializations for other character sets specify a static association. These types satisfy the `Character` concept and are default constructible, copy constructible, copy assignable, and equality comparable. Member functions provide access to the code point and character set ID values for the represented character. Default constructed objects represent a null character using a zero initialized code point value.

Objects with different character set type are not equality comparable with the exception that objects with a static character set type of `any_character_set` are comparable with objects with any static character set type. In this case, objects compare equally if and only if their character set ID and code point values match. Equality comparison between objects with different static character set type is not implemented to avoid potentially costly unintended implicit transcoding between character sets.

```

template<CharacterSet CST>
class character {
public:
    using character_set_type = CST;
    using code_point_type = code_point_type_t<character_set_type>;

    character() = default;
    explicit character(code_point_type code_point) noexcept;

    friend bool operator==(const character &lhs,
                           const character &rhs) noexcept;
    friend bool operator!=(const character &lhs,
                           const character &rhs) noexcept;

    void set_code_point(code_point_type code_point) noexcept;
    code_point_type get_code_point() const noexcept;

    static character_set_id get_character_set_id();

private:
    code_point_type code_point; // exposition only
};

template<>
class character<any_character_set> {
public:
    using character_set_type = any_character_set;
    using code_point_type = code_point_type_t<character_set_type>;

    character() = default;
    explicit character(code_point_type code_point) noexcept;
    character(character_set_id cs_id, code_point_type code_point) noexcept;

    friend bool operator==(const character &lhs,
                           const character &rhs) noexcept;
    friend bool operator!=(const character &lhs,
                           const character &rhs) noexcept;

    void set_code_point(code_point_type code_point) noexcept;
    code_point_type get_code_point() const noexcept;

```

```

void set_character_set_id(character_set_id new_cs_id) noexcept;
character_set_id get_character_set_id() const noexcept;

private:
    character_set_id cs_id;    // exposition only
    code_point_type code_point; // exposition only
};

template<CharacterSet CST>
    bool operator==(const character<any_character_set> &lhs,
                    const character<CST> &rhs);
template<CharacterSet CST>
    bool operator==(const character<CST> &lhs,
                    const character<any_character_set> &rhs);
template<CharacterSet CST>
    bool operator!=(const character<any_character_set> &lhs,
                    const character<CST> &rhs);
template<CharacterSet CST>
    bool operator!=(const character<CST> &lhs,
                    const character<any_character_set> &rhs);

```

Encodings

- [class trivial_encoding_state](#)
- [class trivial_encoding_state_transition](#)
- [Class basic_execution_character_encoding](#)
- [Class basic_execution_wide_character_encoding](#)
- [Class iso_10646_wide_character_encoding](#)
- [Class utf8_encoding](#)
- [Class utf8bom_encoding](#)
- [Class utf16_encoding](#)
- [Class utf16be_encoding](#)
- [Class utf16le_encoding](#)
- [Class utf16bom_encoding](#)
- [Class utf32_encoding](#)
- [Class utf32be_encoding](#)
- [Class utf32le_encoding](#)
- [Class utf32bom_encoding](#)
- [Encoding type aliases](#)

Class trivial_encoding_state

The `trivial_encoding_state` class is an empty class used by stateless encodings to implement the parts of the generic encoding interfaces necessary to support stateful encodings.

```
class trivial_encoding_state {};
```

Class trivial_encoding_state_transition

The `trivial_encoding_state_transition` class is an empty class used by stateless encodings to implement the parts of the generic encoding interfaces necessary to support stateful encodings that support non-code-point encoding code unit sequences.

```
class trivial_encoding_state_transition {};
```

Class basic_execution_character_encoding

The `basic_execution_character_encoding` class implements support for the encoding used for ordinary string literals limited to support for the basic execution character set as defined in [lex.charset]p3 of the C++ standard.

This encoding is trivial, stateless, fixed width, supports random access decoding, and has a code unit of type `char`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```
class basic_execution_character_encoding {
```

```

public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<basic_execution_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 1;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode_state_transition(state_type &state,
                                                    CUIT &out,
                                                    const state_transition_type &stt,
                                                    int &encoded_code_units)

        noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode(state_type &state,
                                    CUIT &out,
                                    character_type c,
                                    int &encoded_code_units)

        noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static decode_status decode(state_type &state,
                                    CUIT &in_next,
                                    CUST in_end,
                                    character_type &c,
                                    int &decoded_code_units)

        noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static decode_status rdecode(state_type &state,
                                    CUIT &in_next,
                                    CUST in_end,
                                    character_type &c,
                                    int &decoded_code_units)

        noexcept(/* implementation defined */);
};

```

Class `basic_execution_wide_character_encoding`

The `basic_execution_wide_character_encoding` class implements support for the encoding used for wide string literals limited to support for the basic execution wide-character set as defined in [lex.charset]p3 of the C++ standard.

This encoding is trivial, stateless, fixed width, supports random access decoding, and has a code unit of type `wchar_t`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```

class basic_execution_wide_character_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<basic_execution_wide_character_set>;
    using code_unit_type = wchar_t;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 1;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode_state_transition(state_type &state,
                                                    CUIT &out,
                                                    const state_transition_type &stt,
                                                    int &encoded_code_units)

        noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode(state_type &state,

```

```

        CUIT &out,
        character_type c,
        int &encoded_code_units)
noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
requires ranges::ForwardIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
static decode_status decode(state_type &state,
    CUIT &in_next,
    CUST in_end,
    character_type &c,
    int &decoded_code_units)
noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
requires ranges::ForwardIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
static decode_status rdecode(state_type &state,
    CUIT &in_next,
    CUST in_end,
    character_type &c,
    int &decoded_code_units)
noexcept(/* implementation defined */);
};

```

Class iso_10646_wide_character_encoding

The `iso_10646_wide_character_encoding` class is only defined when the `__STDC_ISO_10646__` macro is defined.

The `iso_10646_wide_character_encoding` class implements support for the encoding used for wide string literals when that encoding uses the Unicode character set and `wchar_t` is large enough to store the code point values of all characters defined by the version of the Unicode standard indicated by the value of the `__STDC_ISO_10646__` macro as specified in [cpp.predefined]p2 of the C++ standard.

This encoding is trivial, stateless, fixed width, supports random access decoding, and has a code unit of type `wchar_t`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```

#if defined(__STDC_ISO_10646__)
class iso_10646_wide_character_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = wchar_t;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 1;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode_state_transition(state_type &state,
        CUIT &out,
        const state_transition_type &stt,
        int &encoded_code_units)
    noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode(state_type &state,
        CUIT &out,
        character_type c,
        int &encoded_code_units)
    noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status decode(state_type &state,
        CUIT &in_next,
        CUST in_end,
        character_type &c,
        int &decoded_code_units)
    noexcept(/* implementation defined */);
};

```

```

template<CodeUnitIterator CUIT, typename CUST>
requires ranges::ForwardIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
static decode_status rdecode(state_type &state,
                             CUIT &in_next,
                             CUST in_end,
                             character_type &c,
                             int &decoded_code_units)
noexcept(/* implementation defined */);
};
#endif // __STDC_ISO_10646__

```

Class `utf8_encoding`

The `utf8_encoding` class implements support for the Unicode UTF-8 encoding.

This encoding is stateless, variable width, supports bidirectional decoding, and has a code unit of type `char`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```

class utf8_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<std::make_unsigned_t<code_unit_type>> CUIT>
    static encode_status encode_state_transition(state_type &state,
                                                CUIT &out,
                                                const state_transition_type &stt,
                                                int &encoded_code_units)
    noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<std::make_unsigned_t<code_unit_type>> CUIT>
    static encode_status encode(state_type &state,
                               CUIT &out,
                               character_type c,
                               int &encoded_code_units)
    noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status decode(state_type &state,
                               CUIT &in_next,
                               CUST in_end,
                               character_type &c,
                               int &decoded_code_units)
    noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status rdecode(state_type &state,
                                 CUIT &in_next,
                                 CUST in_end,
                                 character_type &c,
                                 int &decoded_code_units)
    noexcept(/* implementation defined */);
};

```

Class `utf8bom_encoding`

The `utf8bom_encoding` class implements support for the Unicode UTF-8 encoding with a byte order mark (BOM).

This encoding is stateful, variable width, supports bidirectional decoding, and has a code unit of type `char`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

This encoding defines a state transition class that enables forcing or suppressing the encoding of a BOM, or influencing whether a decoded BOM code unit sequence represents a BOM or a code point.

```
class utf8bom_encoding_state {
    /* implementation-defined */
};

class utf8bom_encoding_state_transition {
public:
    static utf8bom_encoding_state_transition to_initial_state() noexcept;
    static utf8bom_encoding_state_transition to_bom_written_state() noexcept;
    static utf8bom_encoding_state_transition to_assume_bom_written_state() noexcept;
};

class utf8bom_encoding {
public:
    using state_type = utf8bom_encoding_state;
    using state_transition_type = utf8bom_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<std::make_unsigned_t<code_unit_type>> CUIT>
        static encode_status encode_state_transition(state_type &state,
                                                    CUIT &out,
                                                    const state_transition_type &stt,
                                                    int &encoded_code_units)
        noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<std::make_unsigned_t<code_unit_type>> CUIT>
        static encode_status encode(state_type &state,
                                    CUIT &out,
                                    character_type c,
                                    int &encoded_code_units)
        noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static decode_status decode(state_type &state,
                                    CUIT &in_next,
                                    CUST in_end,
                                    character_type &c,
                                    int &decoded_code_units)
        noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static decode_status rdecode(state_type &state,
                                    CUIT &in_next,
                                    CUST in_end,
                                    character_type &c,
                                    int &decoded_code_units)
        noexcept(/* implementation defined */);
};
```

Class `utf16_encoding`

The `utf16_encoding` class implements support for the Unicode UTF-16 encoding.

This encoding is stateless, variable width, supports bidirectional decoding, and has a code unit of type `char16_t`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```
class utf16_encoding {
public:
```



```

using state_type = trivial_encoding_state;
using state_transition_type = trivial_encoding_state_transition;
using character_type = character<unicode_character_set>;
using code_unit_type = char16_t;

static constexpr int min_code_units = 1;
static constexpr int max_code_units = 2;

static const state_type& initial_state() noexcept;

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode_state_transition(state_type &state,
                                                CUIT &out,
                                                const state_transition_type &stt,
                                                int &encoded_code_units)
    noexcept(/* implementation defined */);

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode(state_type &state,
                                CUIT &out,
                                character_type c,
                                int &encoded_code_units;
    noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status decode(state_type &state,
                                CUIT &in_next,
                                CUST in_end,
                                character_type &c,
                                int &decoded_code_units)
    noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status rdecode(state_type &state,
                                CUIT &in_next,
                                CUST in_end,
                                character_type &c,
                                int &decoded_code_units)
    noexcept(/* implementation defined */);
};

```

Class utf16be_encoding

The `utf16be_encoding` class implements support for the Unicode UTF-16 big-endian encoding.

This encoding is stateless, variable width, supports bidirectional decoding, and has a code unit of type `char`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```
class utf16be_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 2;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode_state_transition(state_type &state,
                                                    CUIT &out,
                                                    const state_transition_type &stt,
                                                    int &encoded_code_units)
            noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode(state_type &state,
                                    CUIT &out,
                                    character_type c,
```

```

        int &encoded_code_units)
noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
requires ranges::ForwardIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
static decode_status decode(state_type &state,
                           CUIT &in_next,
                           CUST in_end,
                           character_type &c,
                           int &decoded_code_units)
noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
requires ranges::ForwardIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
static decode_status rdecode(state_type &state,
                             CUIT &in_next,
                             CUST in_end,
                             character_type &c,
                             int &decoded_code_units)
noexcept(/* implementation defined */);
};

```

Class utf16le_encoding

The `utf16le_encoding` class implements support for the Unicode UTF-16 little-endian encoding.

This encoding is stateless, variable width, supports bidirectional decoding, and has a code unit of type `char`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```

class utf16le_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 2;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode_state_transition(state_type &state,
                                                CUIT &out,
                                                const state_transition_type &stt,
                                                int &encoded_code_units)
    noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode(state_type &state,
                               CUIT &out,
                               character_type c,
                               int &encoded_code_units)
    noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status decode(state_type &state,
                               CUIT &in_next,
                               CUST in_end,
                               character_type &c,
                               int &decoded_code_units)
    noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status rdecode(state_type &state,
                                 CUIT &in_next,
                                 CUST in_end,

```

```

        character_type &c,
        int &decoded_code_units)
    noexcept(/* implementation defined */);
};

```

Class utf16bom_encoding

The `utf16bom_encoding` class implements support for the Unicode UTF-16 encoding with a byte order mark (BOM).

This encoding is stateful, variable width, supports bidirectional decoding, and has a code unit of type `char`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

This encoding defines a state transition class that enables forcing or suppressing the encoding of a BOM, or influencing whether a decoded BOM code unit sequence represents a BOM or a code point.

```

class utf16bom_encoding_state {
    /* implementation-defined */
};

class utf16bom_encoding_state_transition {
public:
    static utf16bom_encoding_state_transition to_initial_state() noexcept;
    static utf16bom_encoding_state_transition to_bom_written_state() noexcept;
    static utf16bom_encoding_state_transition to_be_bom_written_state() noexcept;
    static utf16bom_encoding_state_transition to_le_bom_written_state() noexcept;
    static utf16bom_encoding_state_transition to_assume_bom_written_state() noexcept;
    static utf16bom_encoding_state_transition to_assume_be_bom_written_state() noexcept;
    static utf16bom_encoding_state_transition to_assume_le_bom_written_state() noexcept;
};

class utf16bom_encoding {
public:
    using state_type = utf16bom_encoding_state;
    using state_transition_type = utf16bom_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 2;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode_state_transition(state_type &state,
                                                    CUIT &out,
                                                    const state_transition_type &stt,
                                                    int &encoded_code_units)

        noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode(state_type &state,
                                    CUIT &out,
                                    character_type c,
                                    int &encoded_code_units)

        noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static decode_status decode(state_type &state,
                                    CUIT &in_next,
                                    CUST in_end,
                                    character_type &c,
                                    int &decoded_code_units)

        noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static decode_status rdecode(state_type &state,
                                    CUIT &in_next,
                                    CUST in_end,
                                    character_type &c,
                                    int &decoded_code_units)

        noexcept(/* implementation defined */);

```

```
};
```

Class utf32_encoding

The `utf32_encoding` class implements support for the Unicode UTF-32 encoding.

This encoding is trivial, stateless, fixed width, supports random access decoding, and has a code unit of type `char32_t`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```
class utf32_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char32_t;

    static constexpr int min_code_units = 1;
    static constexpr int max_code_units = 1;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode_state_transition(state_type &state,
                                                    CUIT &out,
                                                    const state_transition_type &stt,
                                                    int &encoded_code_units)
            noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode(state_type &state,
                                    CUIT &out,
                                    character_type c,
                                    int &encoded_code_units)
            noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static decode_status decode(state_type &state,
                                    CUIT &in_next,
                                    CUST in_end,
                                    character_type &c,
                                    int &decoded_code_units)
            noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static decode_status rdecode(state_type &state,
                                    CUIT &in_next,
                                    CUST in_end,
                                    character_type &c,
                                    int &decoded_code_units)
            noexcept(/* implementation defined */);
};
```

Class utf32be_encoding

The `utf32be_encoding` class implements support for the Unicode UTF-32 big-endian encoding.

This encoding is stateless, fixed width, supports random access decoding, and has a code unit of type `char`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```
class utf32be_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;
```

```

static constexpr int min_code_units = 4;
static constexpr int max_code_units = 4;

static const state_type& initial_state() noexcept;

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode_state_transition(state_type &state,
                                                CUIT &out,
                                                const state_transition_type &stt,
                                                int &encoded_code_units)

    noexcept(/* implementation defined */);

template<CodeUnitOutputIterator<code_unit_type> CUIT>
    static encode_status encode(state_type &state,
                                CUIT &out,
                                character_type c,
                                int &encoded_code_units)

    noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status decode(state_type &state,
                                CUIT &in_next,
                                CUST in_end,
                                character_type &c,
                                int &decoded_code_units)

    noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status rdecode(state_type &state,
                                CUIT &in_next,
                                CUST in_end,
                                character_type &c,
                                int &decoded_code_units)

    noexcept(/* implementation defined */);
};

```

Class utf32le_encoding

The `utf32le_encoding` class implements support for the Unicode UTF-32 little-endian encoding.

This encoding is stateless, fixed width, supports random access decoding, and has a code unit of type `char`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

```

class utf32le_encoding {
public:
    using state_type = trivial_encoding_state;
    using state_transition_type = trivial_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 4;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode_state_transition(state_type &state,
                                                    CUIT &out,
                                                    const state_transition_type &stt,
                                                    int &encoded_code_units)

        noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode(state_type &state,
                                    CUIT &out,
                                    character_type c,
                                    int &encoded_code_units)

        noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()

```

```

        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status decode(state_type &state,
                               CUIT &in_next,
                               CUST in_end,
                               character_type &c,
                               int &decoded_code_units)
    noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
    requires ranges::ForwardIterator<CUIT>()
        && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
        && ranges::Sentinel<CUST, CUIT>()
    static decode_status rdecode(state_type &state,
                                 CUIT &in_next,
                                 CUST in_end,
                                 character_type &c,
                                 int &decoded_code_units)
    noexcept(/* implementation defined */);
};

```

Class utf32bom_encoding

The `utf32bom_encoding` class implements support for the Unicode UTF-32 encoding with a byte order mark (BOM).

This encoding is stateful, variable width, supports bidirectional decoding, and has a code unit of type `char`.

Errors that occur during encoding and decoding operations are reported via the `encode_status` and `decode_status` return types. Exceptions are not directly thrown, but may propagate from operations performed on the dependent code unit iterator.

This encoding defines a state transition class that enables forcing or suppressing the encoding of a BOM, or influencing whether a decoded BOM code unit sequence represents a BOM or a code point.

```

class utf32bom_encoding_state {
    /* implementation-defined */
};

class utf32bom_encoding_state_transition {
public:
    static utf32bom_encoding_state_transition to_initial_state() noexcept;
    static utf32bom_encoding_state_transition to_bom_written_state() noexcept;
    static utf32bom_encoding_state_transition to_be_bom_written_state() noexcept;
    static utf32bom_encoding_state_transition to_le_bom_written_state() noexcept;
    static utf32bom_encoding_state_transition to_assume_bom_written_state() noexcept;
    static utf32bom_encoding_state_transition to_assume_be_bom_written_state() noexcept;
    static utf32bom_encoding_state_transition to_assume_le_bom_written_state() noexcept;
};

class utf32bom_encoding {
public:
    using state_type = utf32bom_encoding_state;
    using state_transition_type = utf32bom_encoding_state_transition;
    using character_type = character<unicode_character_set>;
    using code_unit_type = char;

    static constexpr int min_code_units = 4;
    static constexpr int max_code_units = 4;

    static const state_type& initial_state() noexcept;

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode_state_transition(state_type &state,
                                                      CUIT &out,
                                                      const state_transition_type &stt,
                                                      int &encoded_code_units)
        noexcept(/* implementation defined */);

    template<CodeUnitOutputIterator<code_unit_type> CUIT>
        static encode_status encode(state_type &state,
                                    CUIT &out,
                                    character_type c,
                                    int &encoded_code_units)
        noexcept(/* implementation defined */);

    template<CodeUnitIterator CUIT, typename CUST>
        requires ranges::ForwardIterator<CUIT>()
            && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
            && ranges::Sentinel<CUST, CUIT>()
        static decode_status decode(state_type &state,

```

```

        CUIT &in_next,
        CUST in_end,
        character_type &c,
        int &decoded_code_units)
noexcept(/* implementation defined */);

template<CodeUnitIterator CUIT, typename CUST>
requires ranges::ForwardIterator<CUIT>()
    && ranges::Convertible<ranges::value_type_t<CUIT>, code_unit_type>()
    && ranges::Sentinel<CUST, CUIT>()
static decode_status rdecode(state_type &state,
        CUIT &in_next,
        CUST in_end,
        character_type &c,
        int &decoded_code_units)
noexcept(/* implementation defined */);
};

```

Encoding type aliases

The `execution_character_encoding`, `execution_wide_character_encoding`, `char8_character_encoding`, `char16_character_encoding`, and `char32_character_encoding` type aliases reflect the implementation defined encodings used for execution, wide execution, UTF-8, `char16_t`, and `char32_t` string literals.

Each of these encodings carries a compatibility requirement with another encoding. Decode compatibility is satisfied when the following criteria is met.

1. Text encoded by the compatibility encoding can be decoded by the aliased encoding.
2. Text encoded by the aliased encoding can be decoded by the compatibility encoding when encoded characters are restricted to members of the character set of the compatibility encoding.

These compatibility requirements allow implementation freedom to use encodings that provide features beyond the minimum requirements imposed on the compatibility encodings by the standard. For example, the encoding aliased by `execution_character_encoding` is allowed to support characters that are not members of the character set of the `basic_execution_character_encoding`

The encoding aliased by `execution_character_encoding` must be decode compatible with the `basic_execution_character_encoding` encoding.

The encoding aliased by `execution_wide_character_encoding` must be decode compatible with the `basic_execution_wide_character_encoding` encoding.

The encoding aliased by `char8_character_encoding` must be decode compatible with the `utf8_encoding` encoding.

The encoding aliased by `char16_character_encoding` must be decode compatible with the `utf16_encoding` encoding.

The encoding aliased by `char32_character_encoding` must be decode compatible with the `utf32_encoding` encoding.

```

using execution_character_encoding = /* implementation-defined */ ;
using execution_wide_character_encoding = /* implementation-defined */ ;
using char8_character_encoding = /* implementation-defined */ ;
using char16_character_encoding = /* implementation-defined */ ;
using char32_character_encoding = /* implementation-defined */ ;

```

Text Iterators

- [Class template `itext\_iterator`](#)
- [Class template `itext\_sentinel`](#)
- [Class template `otext\_iterator`](#)
- [make `otext\_iterator`](#)

Class template `itext_iterator`

Objects of `itext_iterator` class template specialization type provide a standard iterator interface for enumerating the characters encoded by the associated encoding `ET` in the code unit sequence exposed by the associated view. These types satisfy the `TextInputIterator` concept and are default constructible, copy and move constructible, copy and move assignable, and equality comparable.

These types also conditionally satisfy `ranges::ForwardIterator`, `ranges::BidirectionalIterator`, and `ranges::RandomAccessIterator` depending on traits of the associated encoding `ET` and view `VT` as described in the following table.

	then <code>itext_iterator<ET, VT></code>	and <code>itext_iterator<ET,</code>
--	---	---

When ET and ranges::iterator_t<VT> satisfy ...	satisfies ...	VT::iterator_category is ...
ranges::InputIterator<ranges::iterator_t<VT>> && ! ranges::ForwardIterator<ranges::iterator_t<VT>> && TextForwardDecoder<ET, /* implementation-defined */ > (With an internal adapter to provide forward iterator semantics over the input iterator)	ranges::InputIterator	std::input_iterator_tag
TextForwardDecoder<ET, ranges::iterator_t<VT>>	ranges::ForwardIterator	std::forward_iterator_tag
TextBidirectionalDecoder<ET, ranges::iterator_t<VT>>	ranges::BidirectionalIterator	std::bidirectional_iterator_tag
TextRandomAccessDecoder<ET, ranges::iterator_t<VT>>	ranges::RandomAccessIterator	std::random_access_iterator_tag

Member functions provide access to the stored encoding state, the underlying code unit iterator, and the underlying code unit range for the current character. The underlying code unit range is returned with an implementation defined type that satisfies ranges::View. The is_ok member function returns true if the iterator is dereferenceable as a result of having successfully decoded a code point (This predicate is used to distinguish between an input iterator that just successfully decoded the last code point in the code unit stream as compared to one that was advanced after having done so; in both cases, the underlying code unit input iterator will compare equal to the end iterator).

The error_occurred and get_error member functions enable retrieving information about errors that occurred during decoding operations. if a call to error_occurred returns false, then it is guaranteed that a dereference operation will not throw an exception; assuming a non-singular iterator that is not past the end.

The look_ahead_range member function is provided only when the underlying code unit iterator is an input iterator; it provides access to code units that were read from the code unit iterator, but were not (yet) used to decode a character. Generally such look ahead only occurs when an invalid code unit sequence is encountered.

```

template<TextEncoding ET,
        ranges::View VT,
        TextErrorPolicy TEP = text_default_error_policy>
requires TextForwardDecoder<
    ET,
    /* implementation-defined */>()
class itext_iterator {
public:
    using encoding_type = ET;
    using view_type = VT;
    using error_policy = TEP;
    using state_type = typename encoding_type::state_type;

    using iterator = ranges::iterator_t<std::add_const_t<view_type>>>;
    using iterator_category = /* implementation-defined */;
    using value_type = character_type_t<encoding_type>;
    using reference = value_type;
    using pointer = std::add_const_t<value_type>*;
    using difference_type = ranges::difference_type_t<iterator>;

    itext_iterator();

    itext_iterator(state_type state,
                  const view_type *view,
                  iterator first);

    reference operator*() const noexcept;
    pointer operator->() const noexcept;

    friend bool operator==(const itext_iterator &l, const itext_iterator &r);
    friend bool operator!=(const itext_iterator &l, const itext_iterator &r);

    friend bool operator<(const itext_iterator &l, const itext_iterator &r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();
    friend bool operator>(const itext_iterator &l, const itext_iterator &r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();
    friend bool operator<=(const itext_iterator &l, const itext_iterator &r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();
    friend bool operator>=(const itext_iterator &l, const itext_iterator &r)
        requires TextRandomAccessDecoder<encoding_type, iterator>();

    itext_iterator& operator++();
    itext_iterator& operator++()
        requires TextForwardDecoder<encoding_type, iterator>();
    itext_iterator operator++(int);

    itext_iterator& operator--()
        requires TextBidirectionalDecoder<encoding_type, iterator>();
    itext_iterator operator--(int)

```



```

requires TextBidirectionalDecoder<encoding_type, iterator>();

itext_iterator& operator+=(difference_type n)
    requires TextRandomAccessDecoder<encoding_type, iterator>();
itext_iterator& operator-=(difference_type n)
    requires TextRandomAccessDecoder<encoding_type, iterator>();

friend itext_iterator operator+(itext_iterator l, difference_type n)
    requires TextRandomAccessDecoder<encoding_type, iterator>();
friend itext_iterator operator+(difference_type n, itext_iterator r)
    requires TextRandomAccessDecoder<encoding_type, iterator>();

friend itext_iterator operator-(itext_iterator l, difference_type n)
    requires TextRandomAccessDecoder<encoding_type, iterator>();
friend difference_type operator-(const itext_iterator &l,
                                const itext_iterator &r)
    requires TextRandomAccessDecoder<encoding_type, iterator>();

reference operator[](difference_type n) const
    requires TextRandomAccessDecoder<encoding_type, iterator>();

const state_type& state() const noexcept;

const iterator& base() const noexcept;

/* implementation-defined */ base_range() const noexcept;

/* implementation-defined */ look_ahead_range() const noexcept
    requires ! ranges::ForwardIterator<iterator>();

bool error_occurred() const noexcept;
decode_status get_error() const noexcept;

bool is_ok() const noexcept;

private:
    state_type base_state; // exposition only
    iterator base_iterator; // exposition only
    bool ok; // exposition only
};

```

Class template `itext_sentinel`

Objects of `itext_sentinel` class template specialization type denote the end of a range of text as delimited by a sentinel object for the underlying code unit sequence. These types satisfy the `TextSentinel` concept and are default constructible, copy and move constructible, and copy and move assignable. Member functions provide access to the sentinel for the underlying code unit sequence.

Objects of these types are equality comparable to `itext_iterator` objects that have matching encoding and view types.

```

template<TextEncoding ET,
        ranges::View VT,
        TextErrorPolicy TEP = text_default_error_policy>
class itext_sentinel {
public:
    using view_type = VT;
    using error_policy = TEP;
    using sentinel = ranges::sentinel_t<std::add_const_t<view_type>>;

    itext_sentinel() = default;

    itext_sentinel(sentinel s);

    friend bool operator==(const itext_iterator<ET, VT, TEP> &ti,
                           const itext_sentinel &ts);
    friend bool operator!=(const itext_iterator<ET, VT, TEP> &ti,
                           const itext_sentinel &ts);
    friend bool operator==(const itext_sentinel &ts,
                           const itext_iterator<ET, VT, TEP> &ti);
    friend bool operator!=(const itext_sentinel &ts,
                           const itext_iterator<ET, VT, TEP> &ti);

    const sentinel& base() const noexcept;

private:
    sentinel base_sentinel; // exposition only
};

```

Class template `otext_iterator`

Objects of `otext_iterator` class template specialization type provide a standard iterator interface for encoding characters in the form implemented by the associated encoding `ET`. These types satisfy the `TextOutputIterator` concept and are default constructible, copy and move constructible, and copy and move assignable.

Member functions provide access to the stored encoding state and the underlying code unit output iterator.

The `error_occurred` and `get_error` member functions enable retrieving information about errors that occurred during encoding operations.

```
template<TextEncoding ET,
        CodeUnitOutputIterator<code_unit_type_t<ET>> CUIT,
        TextErrorPolicy TEP = text_default_error_policy>
class otext_iterator {
public:
    using encoding_type = ET;
    using error_policy = TEP;
    using state_type = typename ET::state_type;
    using state_transition_type = typename ET::state_transition_type;

    using iterator = CUIT;
    using iterator_category = std::output_iterator_tag;
    using value_type = character_type_t<encoding_type>;
    using reference = value_type&;
    using pointer = value_type*;
    using difference_type = ranges::difference_type_t<iterator>;

    otext_iterator();

    otext_iterator(state_type state, iterator current);

    otext_iterator& operator*() const noexcept;

    otext_iterator& operator++() noexcept;
    otext_iterator& operator++(int) noexcept;

    otext_iterator& operator=(const state_transition_type &stt);
    otext_iterator& operator=(const character_type_t<encoding_type> &value);

    const state_type& state() const noexcept;

    const iterator& base() const noexcept;

    bool error_occurred() const noexcept;
    encode_status get_error() const noexcept;

private:
    state_type base_state; // exposition only
    iterator base_iterator; // exposition only
};
```

make_otext_iterator

The `make_otext_iterator` functions enable convenient construction of `otext_iterator` objects via type deduction of the underlying code unit output iterator type. Overloads are provided to enable construction with an explicit encoding state or the implicit encoding dependent initial state.

```
template<TextEncoding ET,
        TextErrorPolicy TEP,
        CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
auto make_otext_iterator(typename ET::state_type state, IT out)
-> otext_iterator<ET, IT>;
template<TextEncoding ET,
        CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
auto make_otext_iterator(typename ET::state_type state, IT out)
-> otext_iterator<ET, IT>;
template<TextEncoding ET,
        TextErrorPolicy TEP,
        CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
auto make_otext_iterator(IT out)
-> otext_iterator<ET, IT>;
template<TextEncoding ET,
        CodeUnitOutputIterator<code_unit_type_t<ET>> IT>
auto make_otext_iterator(IT out)
-> otext_iterator<ET, IT>;
```



```

template<typename charT, typename traits, typename Allocator>
basic_text_view(const basic_string<charT, traits, Allocator> &str)
requires ranges::Constructible<code_unit_iterator, const charT *>()
    && ranges::ConvertibleTo<ranges::difference_type_t<code_unit_iterator>,
        typename basic_string<charT, traits, Allocator>::size_type>()
    && ranges::Constructible<view_type,
        code_unit_iterator,
        code_unit_sentinel>();

template<ranges::InputRange Iterable>
basic_text_view(state_type state,
    const Iterable &iterable)
requires ranges::Constructible<code_unit_iterator,
    ranges::iterator_t<const Iterable>>()
    && ranges::Constructible<view_type,
        code_unit_iterator,
        code_unit_sentinel>();

template<ranges::InputRange Iterable>
basic_text_view(const Iterable &iterable)
requires ranges::Constructible<code_unit_iterator,
    ranges::iterator_t<const Iterable>>()
    && ranges::Constructible<view_type,
        code_unit_iterator,
        code_unit_sentinel>();

basic_text_view(iterator first, iterator last)
requires ranges::Constructible<code_unit_iterator,
    decltype(std::declval<iterator>().base())>()
    && ranges::Constructible<view_type,
        code_unit_iterator,
        code_unit_iterator>();

basic_text_view(iterator first, sentinel last)
requires ranges::Constructible<code_unit_iterator,
    decltype(std::declval<iterator>().base())>()
    && ranges::Constructible<view_type,
        code_unit_iterator,
        code_unit_sentinel>();

const view_type& base() const noexcept;

const state_type& initial_state() const noexcept;

iterator begin() const;
iterator end() const
requires std::is_empty<state_type>::value
    && ranges::Iterator<code_unit_sentinel>();
sentinel end() const
requires !std::is_empty<state_type>::value
    || !ranges::Iterator<code_unit_sentinel>();

private:
    state_type base_state; // exposition only
    view_type base_view;   // exposition only
};

```

Text view type aliases

The `text_view`, `wtext_view`, `u8text_view`, `u16text_view` and `u32text_view` type aliases reference an implementation defined specialization of `basic_text_view` for all five of the encodings the standard states must be provided.

The implementation defined view type used for the underlying code unit view type must satisfy `ranges::View` and provide iterators of pointer to the underlying code unit type to contiguous storage. The intent in providing these type aliases is to minimize instantiations of the `basic_text_view` and `itext_iterator` class templates by encouraging use of common view types with underlying code unit views that reference contiguous storage, such as views into objects with a type instantiated from `std::basic_string`. See further discussion in the [View Requirements](#) section.

It is permissible for the `text_view` and `u8text_view` type aliases to reference the same type. This will be the case when the execution character encoding is UTF-8. Attempts to overload functions based on `text_view` and `u8text_view` will result in multiple function definition errors on such implementations.

```

using text_view = basic_text_view<
    execution_character_encoding,
    /* implementation-defined */ >;
using wtext_view = basic_text_view<
    execution_wide_character_encoding,

```

```

        /* implementation-defined */ >;
using u8text_view = basic_text_view<
    char8_character_encoding,
    /* implementation-defined */ >;
using u16text_view = basic_text_view<
    char16_character_encoding,
    /* implementation-defined */ >;
using u32text_view = basic_text_view<
    char32_character_encoding,
    /* implementation-defined */ >;

```

make_text_view

The `make_text_view` functions enable convenient construction of `basic_text_view` objects via implicit selection of a view type for the underlying code unit sequence.

When provided iterators or ranges for contiguous storage, these functions return a `basic_text_view` specialization type that uses the same implementation defined view type as for the `basic_text_view` type aliases as discussed in [Text view type aliases](#)

Overloads are provided to construct `basic_text_view` objects from iterator and sentinel pairs, iterator and difference pairs, and range or `std::basic_string` objects. For each of these overloads, additional overloads are provided to construct the view with an explicit encoding state or with an implicit initial encoding state provided by the encoding `ET`. Each of these overloads requires that the encoding type be explicitly specified.

Additional overloads are provided to construct the view from iterator and sentinel pairs that satisfy `TextInputIterator` and objects of a type that satisfies `TextView`. For these overloads, the encoding type is deduced and the encoding state is implicitly copied from the arguments.

If `make_text_view` is invoked with an rvalue range, then the lifetime of the returned object and all copies of it must end with the full-expression that the `make_text_view` invocation is within. Otherwise, the returned object or its copies will hold iterators into a destructed object resulting in undefined behavior.

```

template<TextEncoding ET,
        TextErrorPolicy TEP,
        ranges::InputIterator IT,
        ranges::Sentinel<IT> ST>
    auto make_text_view(typename ET::state_type state,
                        IT first, ST last)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET,
        ranges::InputIterator IT,
        ranges::Sentinel<IT> ST>
    auto make_text_view(typename ET::state_type state,
                        IT first, ST last)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextErrorPolicy TEP,
        ranges::InputIterator IT,
        ranges::Sentinel<IT> ST>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<IT>>;
    }
    auto make_text_view(typename default_encoding_type_t<ranges::value_type_t<IT>>::state_type state,
                        IT first,
                        ST last)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;

template<ranges::InputIterator IT,
        ranges::Sentinel<IT> ST>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<IT>>;
    }
    auto make_text_view(typename default_encoding_type_t<ranges::value_type_t<IT>>::state_type state,
                        IT first,
                        ST last)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;

template<TextEncoding ET,
        TextErrorPolicy TEP,
        ranges::InputIterator IT,
        ranges::Sentinel<IT> ST>
    auto make_text_view(IT first, ST last)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET,
        ranges::InputIterator IT,

```

```

        ranges::Sentinel<IT> ST>
    auto make_text_view(IT first, ST last)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextErrorPolicy TEP,
        ranges::InputIterator IT,
        ranges::Sentinel<IT> ST>
requires requires () {
    typename default_encoding_type_t<ranges::value_type_t<IT>>;
}
auto make_text_view(IT first,
                    ST last)
-> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;

template<ranges::InputIterator IT,
        ranges::Sentinel<IT> ST>
requires requires () {
    typename default_encoding_type_t<ranges::value_type_t<IT>>;
}
auto make_text_view(IT first,
                    ST last)
-> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;

template<TextEncoding ET,
        TextErrorPolicy TEP,
        ranges::ForwardIterator IT>
auto make_text_view(typename ET::state_type state,
                    IT first,
                    ranges::difference_type_t<IT> n)
-> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET,
        ranges::ForwardIterator IT>
auto make_text_view(typename ET::state_type state,
                    IT first,
                    ranges::difference_type_t<IT> n)
-> basic_text_view<ET, /* implementation-defined */ >;

template<TextErrorPolicy TEP,
        ranges::ForwardIterator IT>
requires requires () {
    typename default_encoding_type_t<ranges::value_type_t<IT>>;
}
auto make_text_view(typename default_encoding_type_t<ranges::value_type_t<IT>>::state_type state,
                    IT first,
                    ranges::difference_type_t<IT> n)
-> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;

template<ranges::ForwardIterator IT>
requires requires () {
    typename default_encoding_type_t<ranges::value_type_t<IT>>;
}
auto make_text_view(typename default_encoding_type_t<ranges::value_type_t<IT>>::state_type state,
                    IT first,
                    ranges::difference_type_t<IT> n)
-> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;

template<TextEncoding ET,
        TextErrorPolicy TEP,
        ranges::ForwardIterator IT>
auto make_text_view(IT first,
                    ranges::difference_type_t<IT> n)
-> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET,
        ranges::ForwardIterator IT>
auto make_text_view(IT first,
                    ranges::difference_type_t<IT> n)
-> basic_text_view<ET, /* implementation-defined */ >;

template<TextErrorPolicy TEP,
        ranges::ForwardIterator IT>
requires requires () {
    typename default_encoding_type_t<ranges::value_type_t<IT>>;
}
auto make_text_view(IT first,
                    ranges::difference_type_t<IT> n)
-> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;

template<ranges::ForwardIterator IT>
requires requires () {
    typename default_encoding_type_t<ranges::value_type_t<IT>>;
}

```

```

    auto make_text_view(IT first,
                        ranges::difference_type_t<IT> n)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<IT>>, /* implementation-defined */ >;

template<TextEncoding ET,
        TextErrorPolicy TEP,
        ranges::InputRange Iterable>
    auto make_text_view(typename ET::state_type state,
                        const Iterable &iterable)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET,
        ranges::InputRange Iterable>
    auto make_text_view(typename ET::state_type state,
                        const Iterable &iterable)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextErrorPolicy TEP,
        ranges::InputRange Iterable>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>;
    }
    auto make_text_view(typename default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<RT>>>::state_type state,
                        const RT &range)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>, /* implementation-defined */ >;

template<ranges::InputRange Iterable>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>;
    }
    auto make_text_view(typename default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<RT>>>::state_type state,
                        const RT &range)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>, /* implementation-defined */ >;

template<TextEncoding ET,
        TextErrorPolicy TEP,
        ranges::InputRange Iterable>
    auto make_text_view(const Iterable &iterable)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextEncoding ET,
        ranges::InputRange Iterable>
    auto make_text_view(const Iterable &iterable)
    -> basic_text_view<ET, /* implementation-defined */ >;

template<TextErrorPolicy TEP,
        ranges::InputRange Iterable>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>;
    }
    auto make_text_view(
        const RT &range)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>, /* implementation-defined */ >;

template<ranges::InputRange Iterable>
    requires requires () {
        typename default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>;
    }
    auto make_text_view(
        const RT &range)
    -> basic_text_view<default_encoding_type_t<ranges::value_type_t<ranges::iterator_t<Iterable>>>, /* implementation-defined */ >;

template<TextErrorPolicy TEP,
        TextInputIterator TIT,
        TextSentinel<TIT> TST>
    auto make_text_view(TIT first, TST last)
    -> basic_text_view<encoding_type_t<TIT>, /* implementation-defined */ >;

template<TextInputIterator TIT,
        TextSentinel<TIT> TST>
    auto make_text_view(TIT first, TST last)
    -> basic_text_view<encoding_type_t<TIT>, /* implementation-defined */ >;

template<TextErrorPolicy TEP,
        TextView TVT>
    TVT make_text_view(TVT tv);

template<TextView TVT>
    TVT make_text_view(TVT tv);

```

Acknowledgements

Thank you to the std-proposals community and especially to Zhihao Yuan, Jeffrey Yasskin, Thiago Macieira, and Nicol Bolas for their initial design feedback.

Thank you to Eric Niebler and Casey Carter for the amazing work they've done designing and advancing the Ranges proposal!

References

- [C++11] "Information technology -- Programming languages -- C++", ISO/IEC 14882:2011.
http://www.iso.org/iso/home/store/catalogue_ics/catalogue_detail_ics.htm?csnumber=50372
- [cmcstl2] Casey Carter and Eric Niebler, An implementation of C++ Extensions for Ranges.
<https://github.com/CaseyCarter/cmcstl2>
- [Concepts] "C++ Extensions for concepts", ISO/IEC technical specification 19217:2015.
http://www.iso.org/iso/home/store/catalogue_tc/catalogue_detail.htm?csnumber=64031
- [N2249] Lawrence Crowl, "New Character Types in C++", N2249, 2007.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2249.html>
- [N2442] Lawrence Crowl and Beman Dawes, "Raw and Unicode String Literals; Unified Proposal (Rev. 2)", N2442, 2007.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2442.htm>
- [N3350] Jeffrey Yasskin, "A minimal std::range>Iter>", N3350, 2012.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3350.html>
- [N4560] Eric Niebler and Casey Carter, "Working Draft, C++ Extensions for Ranges", N4560, 2015.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4560.pdf>
- [P0184R0] Eric Niebler, "Generalizing the Range-Based For Loop", P0184R0, 2016.
<http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0184r0.html>
- [P0482R0] Tom Honermann, "char8_t: A type for UTF-8 characters and strings", P0482R0, 2016.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0482r0.html>
- [Text_view] Tom Honermann, Text_view library.
https://github.com/tahonermann/text_view
- [Unicode] "Unicode 8.0.0", 2015.
<http://www.unicode.org/versions/Unicode8.0.0>